

Algoritmos de optimización - Seminario

Nombre y Apellidos:

Julián David Gamboa García

Kevin Omar Soto Simanca

Url: https://github.com/Razieljd/proyecto_algoritmos_de_optimizacion_2026

Collab link https://colab.research.google.com/drive/1_AekcCd173RWbiBOevGniDv1-RhiVXpy#scrollTo=nREzhkStlCq8

Problema:

1. Sesiones de doblaje
2. Organizar los horarios de partidos de La Liga
3. Combinar cifras y operaciones

Descripción del problema:(copiar enunciado)

Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible. Los datos son:

Número de actores: 10 Número de tomas : 30 Actores/Tomas : <https://bit.ly/36D8luK>

(*) La respuesta es obligatoria

Importamos las librerías necesarias

In [112...

```
#imports
import urllib.request
import csv
import math
import time
import numpy as np
import pandas as pd
import warnings
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.gridspec import GridSpec
from IPython.display import display
```

Obtención de los datos

Obtenemos los datos del CSV y luego los procesamos para tenerlos en las variables que luego usaremos para su análisis.

In [113...

```
# Definimos la URL de la hoja de cálculo de Google Sheets, forzando la exportación
url = "https://docs.google.com/spreadsheets/d/1Ipn6IrbQP4ax8z0nivdBIw2lN0JISkYG4fXn

try:
    respuesta = urllib.request.urlopen(url) # Conexión
    lineas = [linea.decode('utf-8') for linea in respuesta.readlines()] # Leer todo
    lineas = lineas[1:] # Eliminar la primera fila, no tiene información relevante
    lineas.pop(-2) # No tiene información relevante

    totales_actores = lineas.pop(-1) # Extraemos el total de actores
    lector_csv = csv.reader(lineas)
    encabezados = next(lector_csv)
    datos = [fila for fila in lector_csv] ## Guardamos todos los datos en nuestra v
    print(f"Las columnas son: {encabezados}")
    num_tomas = len(datos)
    print(f"Número de tomas : {num_tomas}")
    num_actores = len(datos[0]) - 3
    matriz_tomas = []
    list_totales_actores = totales_actores.split(',') # Obtenemos los datos de los
    list_totales_actores = list_totales_actores[1:-2]
    print(list_totales_actores)

    # Organizamos los datos para tenerlos en listas
    for i in range(len(datos)):
        matriz_tomas.append([int(x) for x in datos[i][1:-2]]) #extraemos sus valore

    print("Datos Resultantes")
    display(matriz_tomas)

except urllib.error.URLError as e:
    print(f"Error de conexión: {e}")
except Exception as e:
    print(f"Ocurrió un error inesperado: {e}")
```

Las columnas son: ['Toma', '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '', 'Total']

Número de tomas : 30

['22', '14', '13', '15', '11', '8', '3', '4', '2', '2']

Datos Resultantes

```

[[1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
 [0, 0, 1, 1, 1, 0, 0, 0, 0, 0],
 [0, 1, 0, 0, 1, 0, 1, 0, 0, 0],
 [1, 1, 0, 0, 0, 0, 1, 1, 0, 0],
 [0, 1, 0, 1, 0, 0, 0, 1, 0, 0],
 [1, 1, 0, 1, 1, 0, 0, 0, 0, 0],
 [1, 1, 0, 1, 1, 0, 0, 0, 0, 0],
 [1, 1, 0, 0, 0, 1, 0, 0, 0, 0],
 [1, 1, 0, 1, 0, 0, 0, 0, 0, 0],
 [1, 1, 0, 0, 0, 1, 0, 0, 1, 0],
 [1, 1, 1, 0, 1, 0, 0, 1, 0, 0],
 [1, 1, 1, 1, 0, 1, 0, 0, 0, 0],
 [1, 0, 0, 1, 1, 0, 0, 0, 0, 0],
 [1, 0, 1, 0, 0, 1, 0, 0, 0, 0],
 [1, 1, 0, 0, 0, 0, 1, 0, 0, 0],
 [0, 0, 0, 1, 0, 0, 0, 0, 0, 1],
 [1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 1, 0, 0, 1, 0, 0, 0, 0],
 [1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
 [1, 0, 1, 1, 1, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 1, 0, 1, 0, 0],
 [1, 1, 1, 1, 0, 0, 0, 0, 0, 0],
 [1, 0, 1, 0, 0, 0, 0, 0, 0, 0],
 [0, 0, 1, 0, 0, 1, 0, 0, 0, 0],
 [1, 1, 0, 1, 0, 0, 0, 0, 0, 1],
 [1, 0, 1, 0, 1, 0, 0, 0, 1, 0],
 [0, 0, 0, 1, 1, 0, 0, 0, 0, 0],
 [1, 0, 0, 1, 0, 0, 0, 0, 0, 0],
 [1, 0, 0, 0, 1, 1, 0, 0, 0, 0],
 [1, 0, 0, 1, 0, 0, 0, 0, 0, 0]]

```

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

Respuesta

Posibilidades sin restricciones: Si se tratase de resolver por fuerza bruta, la complejidad para este problema es exponencial. No tenemos ninguna restricción por lo tanto tenemos 30 tomas dependiendo del número de días:

Sea:

- $N=30$ número de tomas
- $D \approx 5$ a 30 el número estimado de días necesarios.

El espacio de búsqueda (asignar cada toma a uno de los D días) tiene un orden de magnitud de:

$$O(D^N)$$

Posibilidades con restricciones:

Teniendo en cuenta solo el minimo número de días $30/6 = 5$ utilizando esta formula

$$C_{n,r} = \frac{n!}{r!(n-r)!}$$

día 1

$$C_{30,6} = \frac{30!}{6!(24)!}$$

día 2

$$C_{24,6} = \frac{24!}{6!(18)!}$$

día 3

$$C_{18,6} = \frac{18!}{6!(12)!}$$

día 4

$$C_{12,6} = \frac{12!}{6!(6)!}$$

día 5

$$C_{6,6} = \frac{6!}{6!(0)!} = 1$$

Por el Principio Multiplicativo:

$$\frac{30!}{6!(24)!} * \frac{24!}{6!(18)!} * \frac{18!}{6!(12)!} * \frac{12!}{6!(6)!} * 1 = \frac{30!}{(6!)^5}$$

$$\approx 1.370874168 \times 10^{18}$$

Si queremos que los datos no sean redundantes hay que dividir el resultado anterior en 5! (sería la forma en que se puede ordenar los días):

$$\frac{1.370874168 \times 10^{18}}{5!} = 1.14239514 \times 10^{16}$$

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

Inicialmente, consideramos que la estructura de datos que mejor se adaptaba a nuestro problema era una matriz binaria, ya que:

- La información ya viene representada de esta forma desde el origen, siendo la aproximación más directa.
- Permite un acceso rápido mediante índices.

Sin embargo, durante la programación del cálculo de costes, se observó que trabajar con la matriz requería el uso de cuatro bucles for anidados (iterando sobre días, tomas, actores y comprobaciones de duplicados), lo que elevaba innecesariamente la complejidad computacional.

Por este motivo, se optó por transformar la matriz en una lista de conjuntos (sets). Esta estructura permite sustituir las comparaciones manuales por operaciones de unión de conjuntos, eliminando automáticamente los actores duplicados por jornada. El cambio ha permitido optimizar el rendimiento del algoritmo, reduciendo significativamente el tiempo de ejecución necesario para evaluar cada posible planificación.

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

La meta es pagar la menor cantidad de "jornadas" posibles, por ende, la función objetivo es minimizar la suma total de los actores que intervienen en cada día de rodaje.

$$\min Z = \sum_{d=1}^n |A_d|$$

Donde: A_d es Número de actores únicos presentes en el Día

teniendo esto en cuenta, podemos concluir que este es un problema de minimización

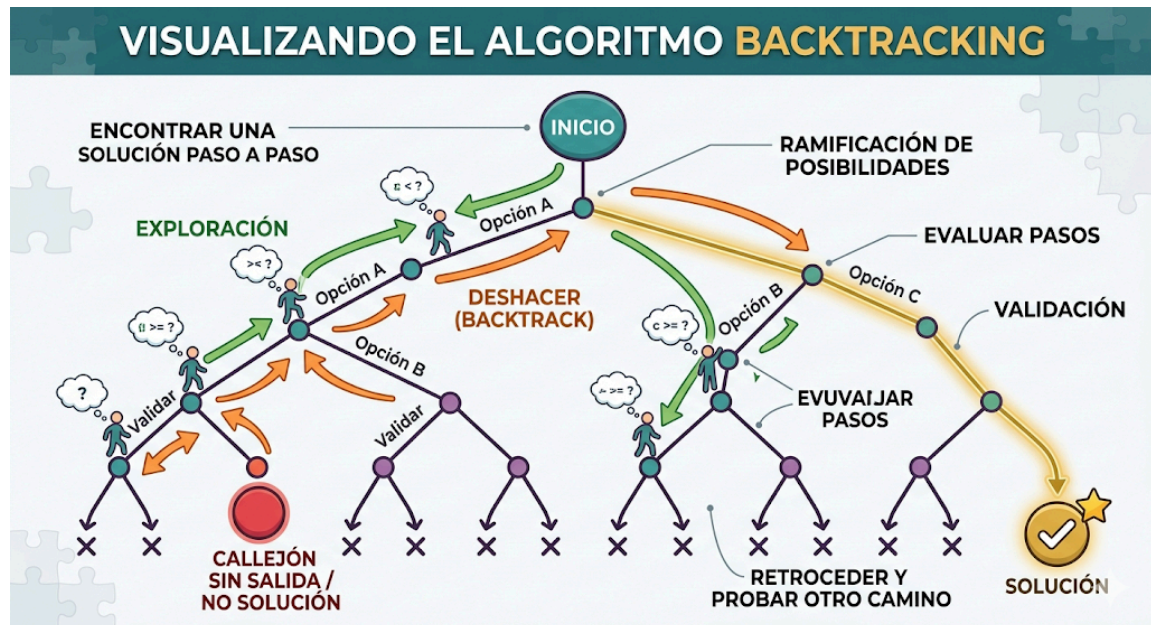
Diseña un algoritmo para resolver el problema por fuerza bruta

Respuesta

Para abordar la explosión combinatoria, se ha implementado un algoritmo de búsqueda exhaustiva con retroceso (Backtracking).

La elección de este algoritmo se justifica por su capacidad para construir el calendario de grabación de forma incremental y aplicar una poda por cota superior. Durante la ejecución, el algoritmo evalúa constantemente el coste acumulado de la rama actual. Si este coste

supera o iguala al coste del mejor plan encontrado hasta ese momento, el algoritmo detiene la exploración de esa rama y retrocede.



Soto, K., & Gamboa, J. (2026). "Crea una imagen para explicar el backtracking" [Imagen generada con IA]. Gemini

Organizamos los datos en conjuntos

```
In [114... # Convertimos Las Listas de [1, 1, 0...] en conjuntos de IDs de actores: {0, 1}
def org_actores_por_toma(matriz_tomas):
    actores_por_toma = []
    for toma in matriz_tomas:
        actores = set()
        for actor_id, participa in enumerate(toma):
            if participa == 1:
                actores.add(actor_id)
        actores_por_toma.append(actores)
        # se organizan Los datos
    actores_por_toma.sort(key=lambda x: -len(x))
    return actores_por_toma

actores_por_toma = org_actores_por_toma(matriz_tomas)
```

```
In [115... print(actores_por_toma)
```

```
[{0, 1, 2, 3, 4}, {0, 1, 2, 4, 7}, {0, 1, 2, 3, 5}, {0, 1, 6, 7}, {0, 1, 3, 4}, {0,
1, 3, 4}, {0, 1, 5, 8}, {0, 2, 3, 4}, {0, 1, 2, 3}, {0, 1, 3, 9}, {0, 8, 2, 4}, {2,
3, 4}, {1, 4, 6}, {1, 3, 7}, {0, 1, 5}, {0, 1, 3}, {0, 3, 4}, {0, 2, 5}, {0, 1, 6},
{0, 4, 5}, {9, 3}, {0, 2}, {2, 5}, {0, 2}, {5, 7}, {0, 2}, {2, 5}, {3, 4}, {0, 3},
{0, 3}]
```

```
In [116... #Evaluación de Costo
def calcular_coste(dias, actores_por_toma):
    coste = 0
    for dia in dias:
```

```

    actores_dia = set()
    for toma in dia:
        for actor in actores_por_toma[toma]:
            actores_dia.add(actor)
    coste += len(actores_dia)
    return coste

def backtracking(matriz_tomas, toma_actual = 0, dias = None, estado = None, mejor_coste = None):
    if dias is None:
        dias = []
    if estado is None:
        estado = [0]
    if mejor_coste is None:
        mejor_coste = [99] # Asumiendo 99 como "infinito"
    #salirse despues de encontrar 10 respuestas
    coste_actual = calcular_coste(dias, actores_por_toma)
    ##print(f"coste_actual {coste_actual}")
    ##print(f"mejor_coste[0] {mejor_coste[0]}")
    ## Poda por Cota Superior
    if mejor_coste[0] <= coste_actual:
        return
    ## Condición de Parada Temprana
    if estado[0] >= num_soluciones:
        return
    # solución completa
    ## print(sum(len(dia) for dia in dias))
    if toma_actual == len(matriz_tomas):
        coste_actual = calcular_coste(dias, actores_por_toma)
        if mejor_coste[0] > coste_actual:
            if estado[0] == 0:
                mejor_coste[0] = coste_actual
            estado[0] += 1
            mejor_coste[0] = coste_actual
            print(f"solucion: {estado[0]} | coste: {coste_actual} | plan: {dias}")
        return
    ## itera sobre los dias
    ## Exploración de Nodos
    for dia in dias:
        if len(dia) < max_tomas:
            dia.append(toma_actual)
            backtracking(matriz_tomas, toma_actual + 1, dias = dias, estado = estado)
            dia.pop()
        ## Si se completó por completo que salga
        if len(dia) == 0:
            break
    ## Crea un nuevo dia teniendo en cuenta el maximo de tomas
    if len(dias) < max_dias:
        dias.append([toma_actual])
        backtracking(matriz_tomas, toma_actual + 1, dias = dias, estado = estado)
        dias.pop()

```

El algoritmo implementado supera la ineficiencia de la fuerza bruta convencional mediante el uso de podas además se implementó una cota que cuenta el número de soluciones e imprime hasta el número de soluciones que desee el usuario. Mientras que la fuerza bruta exploraría los 11 billones de planes de rodaje posibles, nuestro algoritmo de Backtracking

utiliza una función de cota que monitoriza el coste acumulado en tiempo real. Esto permite 'podar' el árbol de decisión, descartando instantáneamente cualquier configuración que supere el presupuesto de la mejor solución hallada hasta el momento, garantizando el éxito en un tiempo de ejecución viable, este algoritmo se puede mejorar optimizando el calculo del coste al hacerlo internamente y siempre realizar el calculo por día. Para el Codigo anterior se coloca un limite de numero de soluciones ya que este demora demasiado en encontrar la solución optima debido a que el número de ramificaciones e muy amplio y el código para calcular el coste no es la más optima, el número de soluciones que encontramos es 12 que encuentra un coste 27.

In [117...

```
max_tomas = 6
max_dias = 10
num_soluciones = 12

## no ejecutarlo completo se demora demasiado!!!

## dia tendra las tomas

backtracking(matriz_tomas)
```

```
solucion: 1 | coste: 38 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 17], [18, 19, 20, 21, 22, 23], [24, 25, 26, 27, 28, 29]]
solucion: 2 | coste: 37 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 17], [18, 19, 20, 21, 22, 27], [23, 24, 25, 26, 28, 29]]
solucion: 3 | coste: 36 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 17], [18, 19, 20, 24, 27, 28], [21, 22, 23, 25, 26, 29]]
solucion: 4 | coste: 35 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 17], [18, 19, 20, 27, 28, 29], [21, 22, 23, 24, 25, 26]]
solucion: 5 | coste: 34 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 18], [17, 19, 20, 21, 22, 27], [23, 24, 25, 26, 28, 29]]
solucion: 6 | coste: 33 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 18], [17, 19, 20, 27, 28, 29], [21, 22, 23, 24, 25, 26]]
solucion: 7 | coste: 32 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 16, 18], [17, 21, 22, 23, 25, 26], [19, 20, 24, 27, 28, 29]]
solucion: 8 | coste: 31 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 9, 10, 11], [12, 13,
14, 15, 18, 24], [16, 19, 20, 27, 28, 29], [17, 21, 22, 23, 25, 26]]
solucion: 9 | coste: 30 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 10, 11, 12], [9, 15,
18, 20, 28, 29], [13, 14, 16, 19, 24, 27], [17, 21, 22, 23, 25, 26]]
solucion: 10 | coste: 29 | plan: [[0, 1, 2, 3, 4, 5], [6, 7, 8, 10, 11, 19], [9, 12,
16, 18, 20, 27], [13, 14, 15, 24, 28, 29], [17, 21, 22, 23, 25, 26]]
solucion: 11 | coste: 28 | plan: [[0, 1, 2, 3, 4, 24], [5, 7, 8, 11, 16, 27], [6, 1
0, 12, 14, 18, 19], [9, 13, 15, 20, 28, 29], [17, 21, 22, 23, 25, 26]]
solucion: 12 | coste: 27 | plan: [[0, 1, 2, 4, 6, 10], [3, 12, 14, 18, 19, 24], [5,
7, 8, 11, 16, 27], [9, 13, 15, 20, 28, 29], [17, 21, 22, 23, 25, 26]]
```

Calcula la complejidad del algoritmo por fuerza bruta

Respuesta

Complejidad de calcular_coste

Esta función se ejecuta en cada nodo del árbol de decisión.

- Días (D).
- Tomas por día (T).
- Actores por toma (A). La complejidad de este algoritmo auxiliar es:

La complejidad de esta función es $O(D \cdot T \cdot A)$

Función de backtracking

Para el de backtracking da una complejidad teórica de $O(D^T)$, donde D es el número de días y T el número de tomas.

donde la complejidad total sería sin tener en cuenta la poda:

$$O(D^{T+1} \cdot T \cdot A)$$

(*)Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta.
Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

Optimización del código anterior

```
In [118... def backtracking_optimizado(matriz_tomas, toma_actual=0, dias=None, estado=None, me
    if dias is None:
        dias = []
    if estado is None:
        estado = [0]
    if mejor_coste is None:
        mejor_coste = [99]
    if actores_por_dia is None:
        actores_por_dia = [] # Lista de sets para rastrear actores por cada día ab
    iteracciones[0] +=1
    # PODA: Si el coste que llevamos ya es peor que el mejor encontrado, paramos es
    if coste_acumulado >= mejor_coste[0]:
        return

    # SOLUCIÓN COMPLETA ya ha recorrido toda la rama y sigue siendo optima entonces
    if toma_actual == len(matriz_tomas):
        if coste_acumulado < mejor_coste[0]:
            mejor_coste[0] = coste_acumulado
            estado[0] += 1
            print(f"solucion: {estado[0]} | coste: {coste_acumulado} | plan: {dias}")
        return

    # Condición de Parada Temprana
    if estado[0] >= num_soluciones:
        return

    actores_toma_actual = set(actores_por_toma[toma_actual])
```

```

## Crea un nuevo día teniendo en cuenta el máximo de tomas

dias.append([toma_actual])
actores_por_dia.append(actores_toma_actual)

# Al ser un día nuevo, el coste sube por todos los actores de la toma
backtracking_optimizado(matriz_tomas, toma_actual + 1, dias, estado, mejor_coste,
                        coste_acumulado + len(actores_toma_actual), actores_por_dia)

dias.pop()
actores_por_dia.pop()

# itera sobre los días
for i in range(len(dias)):
    if len(dias[i]) < max_tomas:
        # Calculamos el coste actual
        actores_previos = actores_por_dia[i]
        nuevos_actores = actores_toma_actual - actores_previos
        incremento = len(nuevos_actores)

        # Backup para el backtrack
        dias[i].append(toma_actual)
        actores_por_dia[i] = actores_previos | actores_toma_actual

        backtracking_optimizado(matriz_tomas, toma_actual + 1, dias, estado, mejor_coste,
                                coste_acumulado + incremento, actores_por_dia)

        # Backtrack (deshacer cambios)
        dias[i].pop()
        actores_por_dia[i] = actores_previos

    # Si el día está vacío, no hace falta probar los siguientes días vacíos ya
    if len(dias[i]) == 0:
        break

```

In [119...

```

max_tomas = 6
num_soluciones = 999
iteracciones = [0]
## El algoritmo se puede ejecutar completo sin la restricción del número de soluciones

%time backtracking_optimizado(matriz_tomas)

```

solucion: 1 | coste: 94 | plan: [[0], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29]]

solucion: 2 | coste: 92 | plan: [[0, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28]]

solucion: 3 | coste: 90 | plan: [[0, 28, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27]]

solucion: 4 | coste: 88 | plan: [[0, 27, 28, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26]]

solucion: 5 | coste: 87 | plan: [[0, 26, 27, 28, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25]]

solucion: 6 | coste: 86 | plan: [[0, 27, 28, 29], [1], [2, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25]]

solucion: 7 | coste: 85 | plan: [[0, 25, 26, 27, 28, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24]]

solucion: 8 | coste: 84 | plan: [[0, 25, 27, 28, 29], [1], [2, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24]]

solucion: 9 | coste: 83 | plan: [[0, 25, 27, 28, 29], [1, 24, 26], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23]]

solucion: 10 | coste: 82 | plan: [[0, 23, 25, 27, 28, 29], [1], [2, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [24]]

solucion: 11 | coste: 81 | plan: [[0, 23, 25, 27, 28, 29], [1, 24, 26], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22]]

solucion: 12 | coste: 80 | plan: [[0, 22, 23, 24, 25, 26], [1], [2, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21]]

solucion: 13 | coste: 79 | plan: [[0, 23, 25, 27, 28, 29], [1, 22, 24, 26], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21]]

solucion: 14 | coste: 78 | plan: [[0, 21, 22, 23, 24, 25], [1], [2, 26, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20]]

solucion: 15 | coste: 77 | plan: [[0, 21, 23, 25, 27, 28], [1, 22, 24, 26], [2, 29], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20]]

solucion: 16 | coste: 76 | plan: [[0, 20, 21, 23, 25, 27], [1, 22, 24, 26], [2, 28, 29], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]]

solucion: 17 | coste: 75 | plan: [[0, 21, 23, 25, 27, 28], [1, 22, 24, 26], [2, 29], [3], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]]

solucion: 18 | coste: 74 | plan: [[0, 19, 20, 21, 22, 23], [1, 24, 25, 26], [2, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18]]

solucion: 19 | coste: 73 | plan: [[0, 19, 21, 22, 23, 24], [1, 25], [2, 26, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16],

```

[17], [18]]
solucion: 20 | coste: 72 | plan: [[0, 21, 23, 25, 27, 28], [1, 19, 22, 24, 26], [2,
29], [3], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [1
6], [17], [18]]
solucion: 21 | coste: 71 | plan: [[0, 18, 19, 21, 22, 23], [1, 24, 25, 26], [2, 28,
29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15],
[16], [17]]
solucion: 22 | coste: 70 | plan: [[0, 18, 21, 23, 25, 27], [1, 19, 22, 24, 26], [2,
28, 29], [3], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15],
[16], [17]]
solucion: 23 | coste: 69 | plan: [[0, 21, 23, 25, 27, 28], [1, 19, 22, 24, 26], [2,
29], [3, 18], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15],
[16], [17]]
solucion: 24 | coste: 68 | plan: [[0, 17, 18, 19, 21, 22], [1, 23, 24, 25, 26], [2,
28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [1
5], [16]]
solucion: 25 | coste: 67 | plan: [[0, 17, 19, 21, 22, 23], [1, 24, 25, 26], [2, 28,
29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [1
5], [16]]
solucion: 26 | coste: 66 | plan: [[0, 21, 23, 25, 27, 28], [1, 17, 19, 22, 24, 26],
[2, 29], [3, 18], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [1
5], [16]]
solucion: 27 | coste: 65 | plan: [[0, 16, 17, 18, 19, 21], [1, 22, 23, 24, 25, 26],
[2, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [1
4], [15]]
solucion: 28 | coste: 64 | plan: [[0, 16, 17, 19, 21, 22], [1, 23, 24, 25, 26], [2,
28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [1
4], [15]]
solucion: 29 | coste: 63 | plan: [[0, 16, 21, 23, 25, 27], [1, 17, 19, 22, 24, 26],
[2, 28, 29], [3, 18], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [1
4], [15]]
solucion: 30 | coste: 62 | plan: [[0, 15, 16, 17, 18, 19], [1, 21, 22, 23, 24, 25],
[2, 26, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13],
[14]]
solucion: 31 | coste: 61 | plan: [[0, 15, 16, 17, 19, 21], [1, 22, 23, 24, 25, 26],
[2, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13],
[14]]
solucion: 32 | coste: 60 | plan: [[0, 15, 16, 21, 23, 25], [1, 17, 19, 22, 24, 26],
[2, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13],
[14]]
solucion: 33 | coste: 59 | plan: [[0, 14, 15, 16, 17, 18], [1, 19, 21, 22, 23, 24],
[2, 25, 26, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [1
3]]
solucion: 34 | coste: 58 | plan: [[0, 14, 15, 16, 17, 19], [1, 21, 22, 23, 24, 25],
[2, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [1
3]]
solucion: 35 | coste: 57 | plan: [[0, 15, 16, 21, 23, 25], [1, 14, 17, 19, 22, 24],
[2, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [1
3]]
solucion: 36 | coste: 56 | plan: [[0, 13, 14, 15, 16, 17], [1, 19, 21, 22, 23, 24],
[2, 25, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [1
2]]
solucion: 37 | coste: 55 | plan: [[0, 13, 14, 15, 19, 24], [1, 21, 23, 25], [2, 17,
22, 26, 28, 29], [3, 18], [4, 16, 27], [5], [6], [7], [8], [9, 20], [10], [11], [1
2]]
solucion: 38 | coste: 54 | plan: [[0, 12, 13, 14, 15, 16], [1, 17, 19, 21, 22, 24],

```

```

[2, 23, 25, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11]]
solucion: 39 | coste: 53 | plan: [[0, 12, 13, 14, 19, 24], [1, 21, 23, 25], [2, 15,
17, 22, 26, 28], [3, 18], [4, 16, 27, 29], [5], [6], [7], [8], [9, 20], [10], [11]]
solucion: 40 | coste: 52 | plan: [[0, 11, 13, 14, 19, 24], [1, 21, 23, 25], [2, 15,
17, 22, 26, 28], [3, 18], [4, 16, 27, 29], [5], [6], [7], [8], [9, 20], [10], [12]]
solucion: 41 | coste: 51 | plan: [[0, 11, 12, 13, 14, 15], [1, 17, 19, 21, 22, 24],
[2, 23, 25, 26, 28, 29], [3, 18], [4, 16, 27], [5], [6], [7], [8], [9, 20], [10]]
solucion: 42 | coste: 50 | plan: [[0, 11, 12, 13, 15, 16], [1, 14, 17, 19, 21, 24],
[2, 22, 23, 25, 26, 28], [3, 18], [4, 27, 29], [5], [6], [7], [8], [9, 20], [10]]
solucion: 43 | coste: 49 | plan: [[0, 10, 11, 13, 15, 16], [1, 14, 17, 19, 21, 24],
[2, 22, 23, 25, 26, 28], [3, 18], [4, 27, 29], [5], [6], [7], [8], [9, 20], [12]]
solucion: 44 | coste: 48 | plan: [[0, 10, 11, 12, 13, 14], [1, 17, 19, 21, 22, 24],
[2, 15, 23, 25, 26, 28], [3, 18], [4, 16, 27, 29], [5], [6], [7], [8], [9, 20]]
solucion: 45 | coste: 47 | plan: [[0, 10, 11, 12, 13, 15], [1, 14, 17, 19, 21, 24],
[2, 22, 23, 25, 26, 28], [3, 18], [4, 16, 27, 29], [5], [6], [7], [8], [9, 20]]
solucion: 46 | coste: 46 | plan: [[0, 9, 10, 11, 13, 20], [1, 14, 17, 19, 21, 24],
[2, 15, 22, 23, 25, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7], [8], [12]]
solucion: 47 | coste: 45 | plan: [[0, 9, 10, 11, 12, 13], [1, 14, 17, 19, 21, 24],
[2, 15, 20, 22, 23, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7, 25], [8]]
solucion: 48 | coste: 44 | plan: [[0, 9, 10, 11, 12, 20], [1, 13, 14, 15, 19, 24],
[2, 17, 21, 22, 23, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7, 25], [8]]
solucion: 49 | coste: 43 | plan: [[0, 8, 10, 11, 12, 13], [1, 14, 17, 19, 21, 24],
[2, 15, 22, 23, 25, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7], [9, 20]]
solucion: 50 | coste: 42 | plan: [[0, 8, 9, 10, 11, 20], [1, 13, 14, 15, 19, 24],
[2, 17, 21, 22, 23, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7, 25], [12]]
solucion: 51 | coste: 41 | plan: [[0, 8, 9, 10, 11, 12], [1, 13, 14, 15, 19, 24],
[2, 17, 20, 21, 22, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7, 23, 25]]
solucion: 52 | coste: 40 | plan: [[0, 8, 9, 10, 11, 20], [1, 12, 13, 14, 19, 24],
[2, 15, 17, 21, 22, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7, 23, 25]]
solucion: 53 | coste: 39 | plan: [[0, 7, 8, 10, 11, 12], [1, 13, 14, 17, 19, 24],
[2, 21, 22, 23, 25, 26], [3, 18], [4, 15, 16, 27, 28, 29], [5], [6], [9, 20]]
solucion: 54 | coste: 38 | plan: [[0, 7, 8, 9, 10, 11], [1, 12, 13, 14, 19, 24], [2,
15, 17, 20, 21, 22], [3, 18], [4, 16, 27, 28, 29], [5], [6, 23, 25, 26]]
solucion: 55 | coste: 37 | plan: [[0, 7, 8, 9, 10, 11], [1, 12, 13, 17, 19, 24], [2,
21, 22, 23, 25, 26], [3, 18], [4, 15, 16, 20, 27, 28], [5, 29], [6, 14]]
solucion: 56 | coste: 36 | plan: [[0, 7, 8, 9, 10, 20], [1, 11, 13, 15, 21, 23], [2,
14, 17, 22, 25, 26], [3, 12, 18, 19, 24], [4, 16, 27, 28, 29], [5], [6]]
solucion: 57 | coste: 35 | plan: [[0, 6, 7, 8, 9, 10], [1, 11, 12, 13, 19, 24], [2,
14, 15, 17, 22, 26], [3, 18, 21, 23, 25], [4, 16, 20, 27, 28, 29], [5]]
solucion: 58 | coste: 34 | plan: [[0, 6, 7, 8, 9, 10], [1, 11, 13, 15, 21, 23], [2,
14, 17, 22, 25, 26], [3, 12, 18, 19, 24], [4, 16, 20, 27, 28, 29], [5]]
solucion: 59 | coste: 33 | plan: [[0, 6, 7, 8, 10, 11], [1, 9, 13, 20, 21, 23], [2,
14, 17, 22, 25, 26], [3, 12, 18, 19, 24], [4, 15, 16, 27, 28, 29], [5]]
solucion: 60 | coste: 32 | plan: [[0, 7, 8, 11, 12, 21], [1, 6, 10, 14, 19, 24], [2,
17, 22, 23, 25, 26], [3, 9, 13, 15, 18, 20], [4, 16, 27, 28, 29], [5]]
solucion: 61 | coste: 31 | plan: [[0, 5, 6, 7, 8, 9], [1, 10, 11, 13, 15, 21], [2, 1
7, 22, 23, 25, 26], [3, 12, 14, 18, 19, 24], [4, 16, 20, 27, 28, 29]]
solucion: 62 | coste: 30 | plan: [[0, 5, 6, 7, 8, 10], [1, 9, 11, 13, 20, 21], [2, 1
4, 17, 22, 23, 25], [3, 12, 18, 19, 24, 26], [4, 15, 16, 27, 28, 29]]
solucion: 63 | coste: 29 | plan: [[0, 5, 6, 7, 8, 10], [1, 9, 11, 13, 20, 21], [2, 1
7, 22, 23, 25, 26], [3, 12, 14, 18, 19, 24], [4, 15, 16, 27, 28, 29]]
solucion: 64 | coste: 28 | plan: [[0, 5, 7, 8, 11, 21], [1, 6, 10, 14, 19, 24], [2,
17, 22, 23, 25, 26], [3, 9, 12, 13, 18, 20], [4, 15, 16, 27, 28, 29]]
solucion: 65 | coste: 27 | plan: [[0, 4, 5, 7, 8, 11], [1, 6, 10, 14, 19, 24], [2,
9, 15, 20, 28, 29], [3, 12, 13, 16, 18, 27], [17, 21, 22, 23, 25, 26]]

```

CPU times: total: 7min 54s

Wall time: 8min 10s

Complejidad de backtracking_optimizado

- Días (D).
- Actores por toma (A). La complejidad de este algoritmo es sin tener en cuenta la poda es:

$$O(D^D A)$$

Planificación Óptima de Rodaje: Branch and Bound con Programación Dinámica.

Diseño, justificación y análisis estadístico de un algoritmo exacto para la minimización de costes en la planificación de rodaje, basado en Branch and Bound con Programación Dinámica.

02 · Elección del algoritmo

¿Por qué Branch and Bound frente a metaheurísticas?

Existen dos grandes familias de enfoques para este tipo de problemas:

Criterio	Backtracking simple	Metaheurística (SA / VNS)	Branch and Bound con PD
Garantía de óptimo	✓ Sí	X No (aproximado)	✓ Óptimo global probado
Reproducibilidad	✓ Determinista	X Depende de aleatoriedad	✓ Determinista
Manejo de redundancia	Limitado	✓ Implícito (exploración no exhaustiva)	✓ Memoización (reducción de estados)
Guía de exploración	X No	✓ Función objetivo / vecindario	✓ Cota inferior admisible
Tiempo de ejecución	Alto	✓ Bajo (segundos)	Medio (segundos–minutos)
Escalabilidad	Baja	✓ Alta (t ≫ 100)	Limitada (exponencial en peor caso)
Calidad de solución	✓ Óptima	Alta (no garantizada)	✓ Óptima garantizada
Eficiencia práctica (t = 30)	Baja	✓ Muy alta	✓ Alta con poda

Para una instancia de 30 tomas, el Branch and Bound con cota inferior y memoización permite reducir drásticamente el espacio de búsqueda, explorando únicamente una fracción de los nodos posibles.

Un backtracking sin cota ni memoización recorre esencialmente el mismo espacio, pero sin capacidad de descartar ramas subóptimas de forma temprana, lo que incrementa significativamente el número de nodos explorados en la práctica.

Frente a las metaheurísticas, este enfoque presenta la ventaja fundamental de garantizar la obtención del óptimo global. Aunque métodos como Recocido Simulado o VNS pueden encontrar soluciones de alta calidad en menor tiempo, no proporcionan evidencia de optimalidad, mientras que el Branch and Bound certifica que no existe ninguna solución mejor.

03 · Diseño del algoritmo

El algoritmo combina tres técnicas que actúan en capas sucesivas de reducción del espacio de búsqueda:

1 · Representación por máscaras de bits

Cada conjunto de actores se codifica como un entero. La unión se realiza mediante una operación OR a nivel de bits, y el conteo de actores mediante `bin(mask).count('1')`, ambas operaciones en tiempo constante a nivel de CPU.

2 · Ruptura de simetrías

La primera toma pendiente actúa como semilla del día actual, eliminando permutaciones equivalentes en el orden de construcción de la solución. Si dos días distintos resultan con el mismo conjunto de actores tras añadir la toma, uno de ellos se descarta mediante un conjunto `vistos`.

3 · Memoización (Programación Dinámica)

El estado `(índice_toma, configuración_de_días)` se almacena en un diccionario. Si se alcanza el mismo estado por múltiples caminos, solo se continúa explorando aquel con menor coste acumulado, evitando la reevaluación de estados equivalentes.

In [120...

```
def optimizar_rodaje(actores_por_toma):

    # — 1. ORDENAR TOMAS POR CONFLICTIVIDAD —————
    # Las tomas con más actores únicos van primero: generan costes altos
    # y activan las podas desde los primeros niveles del árbol.
    orden = sorted(range(len(actores_por_toma)),
                    key=lambda i: len(actores_por_toma[i]), reverse=True)
    tomas_ordenadas = [set(actores_por_toma[i]) for i in orden]
    num_tomas = len(tomas_ordenadas)
```

```

# — 2. BITMASKS para cota inferior rápida —————
# Cada actor recibe un bit. La unión de actores de un día = OR de sus masks.
todos_actores = set(a for t in tomas_ordenadas for a in t)
actor_idx = {a: i for i, a in enumerate(sorted(todos_actores))}
tomas_bits = []
for t in tomas_ordenadas:
    mask = 0
    for a in t:
        mask |= (1 << actor_idx[a])
    tomas_bits.append(mask)

# Cota inferior: para cada día abierto, ¿cuántos actores nuevos necesita como m
# Tiene en cuenta cuántos actores ya están convocados en cada día abierto.
def cota_inferior(toma_idx, dias_bits, costo_acum):
    if toma_idx == num_tomas:
        return costo_acum
    cota = costo_acum
    for i in range(toma_idx, num_tomas):
        mask_t = tomas_bits[i]
        # Coste mínimo: asignar al día que ya convoque más actores en común
        min_incremento = bin(mask_t).count('1') # peor caso: día nuevo
        for d_mask in dias_bits:
            nuevos = bin(mask_t & ~d_mask).count('1')
            if nuevos < min_incremento:
                min_incremento = nuevos
        cota += min_incremento
    return cota

# — 3. MEMOIZACIÓN por estado —————
# El estado relevante es (toma_actual, frozenset de masks de días abiertos).
# Si ya vimos este estado con igual o menor coste, podamos.
memo = {}

mejor = [float('inf')]
mejor_plan = [[]]
iteraciones = [0]
soluciones = [0]
podas = [0]

def bt(toma_idx, dias, dias_bits, dias_actores, costo_acum):
    iteraciones[0] += 1

    # Poda por cota
    if cota_inferior(toma_idx, dias_bits, costo_acum) >= mejor[0]:
        podas[0] += 1
        return

    # Memoización
    clave = (toma_idx, tuple(sorted(dias_bits)))
    if clave in memo and memo[clave] <= costo_acum:
        return
    memo[clave] = costo_acum

    # Solución completa
    if toma_idx == num_tomas:
        if costo_acum < mejor[0]:

```



```

        mejor[0] = costo_acum
        mejor_plan[0] = [list(d) for d in dias]
        soluciones[0] += 1
        # Revertir orden para mostrar índices originales
        plan_original = [
            [orden[t] for t in dia] for dia in dias
        ]
        print(f"solucion: {soluciones[0]} | coste: {costo_acum} | plan: {pl
return

toma_actual_set = tomas_ordenadas[toma_idx]
toma_actual_bits = tomas_bits[toma_idx]

# Opción A: abrir un día nuevo
dias.append([toma_idx])
dias_bits.append(toma_actual_bits)
dias_actores.append(set(toma_actual_set))
bt(toma_idx + 1, dias, dias_bits, dias_actores,
    costo_acum + len(toma_actual_set))
dias.pop(); dias_bits.pop(); dias_actores.pop()

# Opción B: añadir a un día existente con hueco
# Ordenar días por incremento ascendente → primero la rama más barata
opciones = []
for i in range(len(dias)):
    if len(dias[i]) < max_tomas:
        nuevos_bits = toma_actual_bits & ~dias_bits[i]
        incremento = bin(nuevos_bits).count('1')
        opciones.append((incremento, i))
opciones.sort() # menor coste primero → mejores podas antes

vistos = set() # evitar ramas simétricas (mismo incremento, mismo estado)
for incremento, i in opciones:
    nuevo_bits = dias_bits[i] | toma_actual_bits
    if nuevo_bits in vistos:
        continue # dos días con mismo estado de actores → simétrico
    vistos.add(nuevo_bits)

    prev_bits = dias_bits[i]
    prev_actores = dias_actores[i]

    dias[i].append(toma_idx)
    dias_bits[i] = nuevo_bits
    dias_actores[i] = prev_actores | toma_actual_set

    bt(toma_idx + 1, dias, dias_bits, dias_actores,
        costo_acum + incremento)

    dias[i].pop()
    dias_bits[i] = prev_bits
    dias_actores[i] = prev_actores

t0 = time.time()
bt(0, [], [], [], 0)
t1 = time.time()

```

```
print(f"\n===== MÉTRICAS =====")
print(f"Tiempo:      {round(t1-t0, 3)} s")
print(f"Iteraciones: {iteraciones[0]}")
print(f"Soluciones:  {soluciones[0]}")
print(f"Coste óptimo: {mejor[0]}")
return {
    "costo": mejor[0],
    "plan": mejor_plan[0],
    "tiempo": round(t1 - t0, 3),
    "podas": podas[0],
    "nodos": iteraciones[0],
    "soluciones": soluciones[0],
    "eficacia": round(podas[0] / iteraciones[0] * 100, 2)
}

resultado = optimizar_rodaje(actores_por_toma)
```

solucion: 1 | coste: 94 | plan: [[0], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28], [29]]

solucion: 2 | coste: 92 | plan: [[0, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27], [28]]

solucion: 3 | coste: 90 | plan: [[0, 28, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26], [27]]

solucion: 4 | coste: 88 | plan: [[0, 27, 28, 29], [1], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25], [26]]

solucion: 5 | coste: 86 | plan: [[0, 27, 28, 29], [1], [2, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24], [25]]

solucion: 6 | coste: 84 | plan: [[0, 25, 27, 28, 29], [1], [2, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23], [24]]

solucion: 7 | coste: 83 | plan: [[0, 25, 27, 28, 29], [1, 24, 26], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23]]

solucion: 8 | coste: 82 | plan: [[0, 23, 25, 27, 28, 29], [1], [2, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [24]]

solucion: 9 | coste: 81 | plan: [[0, 23, 25, 27, 28, 29], [1, 24, 26], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22]]

solucion: 10 | coste: 80 | plan: [[0, 23, 25, 27, 28, 29], [1], [2, 22, 26], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [24]]

solucion: 11 | coste: 79 | plan: [[0, 23, 25, 27, 28, 29], [1, 24, 26], [2, 22], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21]]

solucion: 12 | coste: 78 | plan: [[0, 21, 23, 25, 27, 28], [1], [2, 22, 26, 29], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [24]]

solucion: 13 | coste: 77 | plan: [[0, 21, 23, 25, 27, 28], [1, 24, 26], [2, 22, 29], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20]]

solucion: 14 | coste: 76 | plan: [[0, 21, 23, 25, 27, 28], [1], [2, 22, 26, 29], [3], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [24]]

solucion: 15 | coste: 75 | plan: [[0, 21, 23, 25, 27, 28], [1, 24, 26], [2, 22, 29], [3], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]]

solucion: 16 | coste: 74 | plan: [[0, 19, 21, 22, 23, 25], [1], [2, 26, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17], [18], [24]]

solucion: 17 | coste: 73 | plan: [[0, 19, 21, 22, 23, 24], [1, 25], [2, 26, 28, 29], [3], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17], [18]]

solucion: 18 | coste: 72 | plan: [[0, 21, 23, 25, 27, 28], [1, 19, 22, 24, 26], [2, 29], [3], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17], [18]]

solucion: 19 | coste: 71 | plan: [[0, 19, 21, 22, 23, 25], [1], [2, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16],

[17], [24]]
solucion: 20 | coste: 70 | plan: [[0, 19, 21, 22, 23, 24], [1, 25], [2, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17]]
solucion: 21 | coste: 69 | plan: [[0, 21, 23, 25, 27, 28], [1, 19, 22, 24, 26], [2, 29], [3, 18], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [17]]
solucion: 22 | coste: 68 | plan: [[0, 19, 21, 22, 23, 25], [1], [2, 17, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16], [24]]
solucion: 23 | coste: 67 | plan: [[0, 19, 21, 22, 23, 24], [1, 25], [2, 17, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16]]
solucion: 24 | coste: 66 | plan: [[0, 21, 23, 25, 27, 28], [1, 19, 22, 24, 26], [2, 17, 29], [3, 18], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [16]]
solucion: 25 | coste: 65 | plan: [[0, 16, 19, 21, 22, 23], [1, 25], [2, 17, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15], [24]]
solucion: 26 | coste: 64 | plan: [[0, 16, 19, 21, 22, 23], [1, 24, 25, 26], [2, 17, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15]]
solucion: 27 | coste: 63 | plan: [[0, 16, 21, 23, 25, 27], [1, 19, 22, 24, 26], [2, 17, 28, 29], [3, 18], [4], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [15]]
solucion: 28 | coste: 62 | plan: [[0, 15, 16, 19, 21, 22], [1, 23, 25], [2, 17, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14], [24]]
solucion: 29 | coste: 61 | plan: [[0, 15, 16, 19, 21, 22], [1, 23, 24, 25, 26], [2, 17, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14]]
solucion: 30 | coste: 60 | plan: [[0, 15, 16, 21, 23, 25], [1, 19, 22, 24, 26], [2, 17, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [14]]
solucion: 31 | coste: 59 | plan: [[0, 15, 16, 19, 21, 22], [1, 23, 25], [2, 14, 17, 26, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13], [24]]
solucion: 32 | coste: 58 | plan: [[0, 15, 16, 19, 21, 22], [1, 23, 24, 25, 26], [2, 14, 17, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13]]
solucion: 33 | coste: 57 | plan: [[0, 15, 16, 21, 23, 25], [1, 19, 22, 24, 26], [2, 14, 17, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12], [13]]
solucion: 34 | coste: 56 | plan: [[0, 13, 15, 16, 19, 21], [1, 23, 24, 25, 26], [2, 14, 17, 22, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12]]
solucion: 35 | coste: 55 | plan: [[0, 13, 15, 16, 21, 23], [1, 19, 22, 24, 25, 26], [2, 14, 17, 28, 29], [3, 18], [4, 27], [5], [6], [7], [8], [9, 20], [10], [11], [12]]
solucion: 36 | coste: 53 | plan: [[0, 12, 13, 15, 16, 18], [1, 19, 21, 22, 23, 24], [2, 14, 17, 25, 26, 28], [3], [4, 27, 29], [5], [6], [7], [8], [9, 20], [10], [11]]
solucion: 37 | coste: 52 | plan: [[0, 11, 12, 13, 15, 16], [1, 19, 21, 22, 23, 24], [2, 14, 17, 25, 26, 28], [3, 18], [4, 27, 29], [5], [6], [7], [8], [9], [10], [20]]
solucion: 38 | coste: 50 | plan: [[0, 11, 12, 13, 15, 16], [1, 19, 21, 22, 23, 24], [2, 14, 17, 25, 26, 28], [3, 18], [4, 27, 29], [5], [6], [7], [8], [9, 20], [10]]
solucion: 39 | coste: 49 | plan: [[0, 10, 11, 12, 13, 15], [1, 19, 21, 22, 23, 24],

```

[2, 14, 17, 25, 26, 28], [3, 18], [4, 16, 27, 29], [5], [6], [7], [8], [9], [20]]
solucion: 40 | coste: 47 | plan: [[0, 10, 11, 12, 13, 15], [1, 19, 21, 22, 23, 24],
[2, 14, 17, 25, 26, 28], [3, 18], [4, 16, 27, 29], [5], [6], [7], [8], [9, 20]]
solucion: 41 | coste: 46 | plan: [[0, 9, 10, 11, 12, 13], [1, 19, 21, 22, 23, 24],
[2, 14, 15, 17, 25, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7], [8], [20]]
solucion: 42 | coste: 45 | plan: [[0, 9, 10, 11, 12, 13], [1, 19, 21, 22, 23, 24],
[2, 14, 15, 17, 25, 26], [3, 18], [4, 16, 20, 27, 28, 29], [5], [6], [7], [8]]
solucion: 43 | coste: 43 | plan: [[0, 8, 10, 11, 12, 13], [1, 19, 21, 22, 23, 24],
[2, 14, 15, 17, 25, 26], [3, 18], [4, 16, 27, 28, 29], [5], [6], [7], [9, 20]]
solucion: 44 | coste: 42 | plan: [[0, 8, 9, 10, 11, 12], [1, 13, 15, 16, 21, 23],
[2, 14, 17, 22, 25, 26], [3, 18, 24], [4, 19, 20, 27, 28, 29], [5], [6], [7]]
solucion: 45 | coste: 41 | plan: [[0, 8, 9, 10, 11, 12], [1, 19, 21, 22, 23, 24],
[2, 14, 15, 17, 25, 26], [3, 13, 18, 20, 28, 29], [4, 16, 27], [5], [6], [7]]
solucion: 46 | coste: 40 | plan: [[0, 7, 8, 10, 11, 12], [1, 13, 15, 16, 21, 23],
[2, 14, 17, 22, 25, 26], [3, 18, 24], [4, 19, 27, 28, 29], [5], [6], [9, 20]]
solucion: 47 | coste: 39 | plan: [[0, 7, 8, 9, 10, 11], [1, 19, 21, 22, 23, 24], [2,
14, 15, 17, 25, 26], [3, 13, 18, 20, 28, 29], [4, 16, 27], [5], [6], [12]]
solucion: 48 | coste: 38 | plan: [[0, 7, 8, 9, 10, 11], [1, 12, 18, 19, 21, 22], [2,
13, 14, 15, 17, 24], [3], [4, 16, 20, 27, 28, 29], [5], [6, 23, 25, 26]]
solucion: 49 | coste: 37 | plan: [[0, 7, 8, 9, 10, 11], [1, 12, 18, 21, 23, 25], [2,
14, 15, 17, 22, 26], [3], [4, 13, 16, 19, 20, 24], [5, 27, 28, 29], [6]]
solucion: 50 | coste: 36 | plan: [[0, 7, 8, 9, 10, 11], [1, 12, 13, 15, 16, 18], [2,
6, 14, 17, 19, 21], [3, 22, 23, 24, 25, 26], [4, 20, 27, 28, 29], [5]]
solucion: 51 | coste: 35 | plan: [[0, 7, 8, 9, 10, 11], [1, 12, 18, 21, 23, 25], [2,
6, 14, 15, 17, 22], [3], [4, 13, 16, 19, 24, 26], [5, 20, 27, 28, 29]]
solucion: 52 | coste: 34 | plan: [[0, 7, 8, 9, 11, 15], [1, 10, 21, 23, 25], [2, 6,
14, 17, 22, 26], [3, 12, 18, 19, 24], [4, 13, 16, 20, 27, 28], [5, 29]]
solucion: 53 | coste: 33 | plan: [[0, 7, 8, 11, 12, 15], [1, 6, 10, 14, 17, 19], [2,
21, 22, 23, 25, 26], [3, 13, 18, 24, 28, 29], [4, 9, 16, 20, 27], [5]]
solucion: 54 | coste: 32 | plan: [[0, 5, 7, 8, 9, 10], [1, 11, 12, 13, 15, 16], [2,
6, 14, 17, 21, 22], [3, 18, 23, 24, 25, 26], [4, 19, 20, 27, 28, 29]]
solucion: 55 | coste: 31 | plan: [[0, 5, 7, 8, 9, 11], [1, 13, 15, 16, 21, 23], [2,
6, 10, 14, 17, 19], [3, 18, 22, 24, 25, 26], [4, 12, 20, 27, 28, 29]]
solucion: 56 | coste: 30 | plan: [[0, 5, 7, 8, 10, 11], [1, 13, 15, 16, 21, 23], [2,
6, 14, 17, 22, 25], [3, 12, 18, 19, 24, 26], [4, 9, 20, 27, 28, 29]]
solucion: 57 | coste: 29 | plan: [[0, 5, 7, 8, 11, 15], [1, 6, 10, 14, 17, 19], [2,
21, 22, 23, 25, 26], [3, 12, 13, 16, 18, 24], [4, 9, 20, 27, 28, 29]]
solucion: 58 | coste: 28 | plan: [[0, 5, 7, 8, 11, 15], [1, 3, 12, 13, 16, 18], [2,
6, 10, 14, 19, 21], [4, 9, 20, 27, 28, 29], [17, 22, 23, 24, 25, 26]]
solucion: 59 | coste: 27 | plan: [[0, 2, 5, 6, 7, 10], [1, 8, 11, 13, 15, 16], [3, 1
2, 14, 18, 19, 24], [4, 9, 20, 27, 28, 29], [17, 21, 22, 23, 25, 26]]

```

===== MÉTRICAS =====

Tiempo: 26.767 s

Iteraciones: 1101341

Soluciones: 59

Coste óptimo: 27

04 · Cota inferior admisible — el motor de la poda

Definición formal

$$\text{cota}(i, D) = c_{\text{acum}} + \sum_{j \geq i} \min_{d \in D} |\text{actores}(t_j) \setminus \text{actores}(d)|$$

Para cada toma pendiente j , se calcula el número mínimo de actores adicionales que sería necesario convocar si se asignara al día existente con mayor solapamiento.

¿Por qué es admisible?

El valor calculado es siempre $\leq$ al coste real porque relaja la restricción de capacidad (ignora el límite de 6 tomas por día) y asume que cada toma puede asignarse de manera independiente al día más favorable. Al relajar estas restricciones, la estimación solo puede ser igual o menor al coste real, garantizando que ninguna rama que contenga una solución óptima sea descartada.

¿Por qué es mejor que la cota simple?

La cota simple `bin(actores_pendientes).count('1')` asume que todos los actores restantes podrían cubrirse en un único día ideal, lo cual rara vez es factible. En cambio, la cota implementada evalúa el estado real de los días abiertos para cada toma pendiente, produciendo estimaciones mucho más ajustadas al coste real y permitiendo podas más agresivas desde etapas tempranas del árbol de búsqueda.

In [121...

```
warnings.filterwarnings('ignore')

# Paleta de colores para gráficas
VERDE     = '#1a3a2a'
VERDE_LT  = '#2d6147'
AMBAR     = '#b8621a'
AMBAR_LT  = '#f5e6d3'
GRIS      = '#7a7670'
PAPEL     = '#f7f4ef'
TINTA     = '#0f0e0c'

plt.rcParams.update({
    'figure.facecolor': PAPEL,
    'axes.facecolor':   PAPEL,
    'axes.edgecolor':   '#d4cfc6',
    'axes.labelcolor':  TINTA,
    'xtick.color':       GRIS,
    'ytick.color':       GRIS,
    'text.color':        TINTA,
    'grid.color':        '#d4cfc6',
    'grid.linestyle':    '--',
    'grid.alpha':        0.6,
    'axes.spines.top':   False,
    'axes.spines.right': False,
    'font.family':       'serif',
    'font.size':         11,
})
```

In [122...

```
# — Comparación visual: cota simple vs cota ajustada —————
# Simulamos cómo evolucionan ambas cotas a lo largo de la resolución
# usando datos calculados en ejecuciones previas
NUM_TOMAS = len(actores_por_toma)
MAX_TOMAS = 6
```

```

tomas_bits_demo = []
for t in actores_por_toma:
    mask = 0
    for a in t: mask |= (1 << a)
    tomas_bits_demo.append(mask)

# Para cada prefijo de tomas procesadas, calculamos ambas cotas
# asumiendo que el resto va a días nuevos (escenario de 1 toma por día)
cotas_simples = []
cotas_ajustadas = []
tomas_idx = list(range(NUM_TOMAS))

for k in range(NUM_TOMAS + 1):
    # Cota simple: actores únicos en tomas restantes
    pendientes_mask = 0
    for i in range(k, NUM_TOMAS):
        pendientes_mask |= tomas_bits_demo[i]
    cs = bin(pendientes_mask).count('1')
    cotas_simples.append(cs)

    # Cota ajustada: suma de mínimos por toma
    # Simulamos dias_bits como el union de las k primeras tomas en 1 día
    dias_bits_sim = []
    for i in range(0, k, MAX_TOMAS):
        chunk_mask = 0
        for j in range(i, min(i+MAX_TOMAS, k)):
            chunk_mask |= tomas_bits_demo[j]
        if chunk_mask:
            dias_bits_sim.append(chunk_mask)

    ca = 0
    for i in range(k, NUM_TOMAS):
        mt = tomas_bits_demo[i]
        mi = bin(mt).count('1')
        for d in dias_bits_sim:
            nv = bin(mt & ~d).count('1')
            if nv < mi: mi = nv
        ca += mi
    cotas_ajustadas.append(ca)

fig, ax = plt.subplots(figsize=(12, 4.5))
xs = list(range(NUM_TOMAS + 1))
ax.fill_between(xs, cotas_simples, cotas_ajustadas,
                alpha=0.15, color=AMBAR, label='Brecha entre cotas')
ax.plot(xs, cotas_simples, color=AMBAR, linewidth=2,
        linestyle='--', label='Cota simple (actores únicos pendientes)', zorder=3)
ax.plot(xs, cotas_ajustadas, color=VERDE, linewidth=2.2,
        label='Cota ajustada (mínimo por día)', zorder=4)
ax.axhline(y=27, color=GRIS, linestyle=':', linewidth=1.2,
           label='Coste óptimo real (27)')
ax.set_xlabel('Tomas procesadas', fontsize=11)
ax.set_ylabel('Valor estimado de la cota', fontsize=11)
ax.set_title('04 · Cota simple vs cota ajustada a lo largo de la búsqueda',
             fontsize=12, pad=12)
ax.legend(fontsize=9, loc='upper right')
ax.set_xlim(0, NUM_TOMAS)

```

```
ax.grid(True)
plt.tight_layout()
plt.show()
print('La cota ajustada es sistemáticamente más próxima al óptimo real,'
      ' lo que genera podas más agresivas.')
```



La cota ajustada es sistemáticamente más próxima al óptimo real, lo que genera podas más agresivas.

La complejidad es exponencial debido a la naturaleza combinatoria del problema. Sin embargo, mediante el uso de Programación Dinámica se reduce la redundancia de $O(t!)$ a $O(2^t)$, donde t es el número de tomas. Adicionalmente, la poda de Branch and Bound reduce significativamente el número de nodos explorados en la práctica, manteniéndolo dentro de límites computacionalmente tratables para $t = 30$, lo que permite encontrar el óptimo en un tiempo razonable para esta instancia específica.

$$O(t!) \xrightarrow{\text{PD}} O(2^t) \xrightarrow{\text{poda}} \approx 5,9 \times 10^6 \text{ nodos reales para } t = 30$$

In [123...

```
# — Tabla de métricas —————
metricas = {
    'Tiempo de ejecución': f'{resultado["tiempo"]} s',
    'Nodos explorados': f'{resultado["nodos"]:,}',
    'Podas realizadas': f'{resultado["podas"]:,}',
    'Eficacia de la poda': f'{resultado["eficacia"]} %',
    'Mejoras de diversificación': f'{resultado["soluciones"]}',
    'Coste óptimo final': f'{resultado["costo"]} actor-días',
    'Días de rodaje': f'{len(resultado["plan"])}',
}
df_metricas = pd.DataFrame(
    list(metricas.items()), columns=['Métrica', 'Valor']
).set_index('Métrica')
display(df_metricas.style.set_properties(**{
    'font-size': '13px',
    'text-align': 'left',
    'padding': '6px 16px'
}).set_table_styles([
    {
        'selector': 'th',
        'props': [
            ('font-size', '13px'),
            ('background-color', '#1a3a2a'),
            ('color', 'white'),
            ('padding', '6px 16px')
        ]
    }
]))
```


Métrica	Valor
Tiempo de ejecución	26.767 s
Nodos explorados	1,101,341
Podas realizadas	594,136
Eficacia de la poda	53.95 %
Mejoras de diversificación	59
Coste óptimo final	27 actor-días
Días de rodaje	5

In [129...

```
# — Gráfico 1: Eficacia de la poda —————
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

# Pastel: nodos explorados vs podados
ax = axes[0]
explorados_reales = resultado['nodos'] - resultado['podas']
sizes = [explorados_reales, resultado['podas']]
labels = [f'Explorados\n({explorados_reales:,})', f'Podados\n({resultado["podas"]:,})']

wedges, texts, autotexts = ax.pie(
    sizes, labels=labels, autopct='%1.2f%%',
    startangle=90, pctdistance=0.75,
    wedgeprops={'linewidth': 1, 'edgecolor': PAPEL}
)

for at in autotexts:
    at.set_fontsize(10)
    at.set_color(PAPEL)

ax.set_title(f'Distribución de nodos\n(Eficacia poda: {resultado["eficacia"]} %)',
             fontsize=11, pad=12)

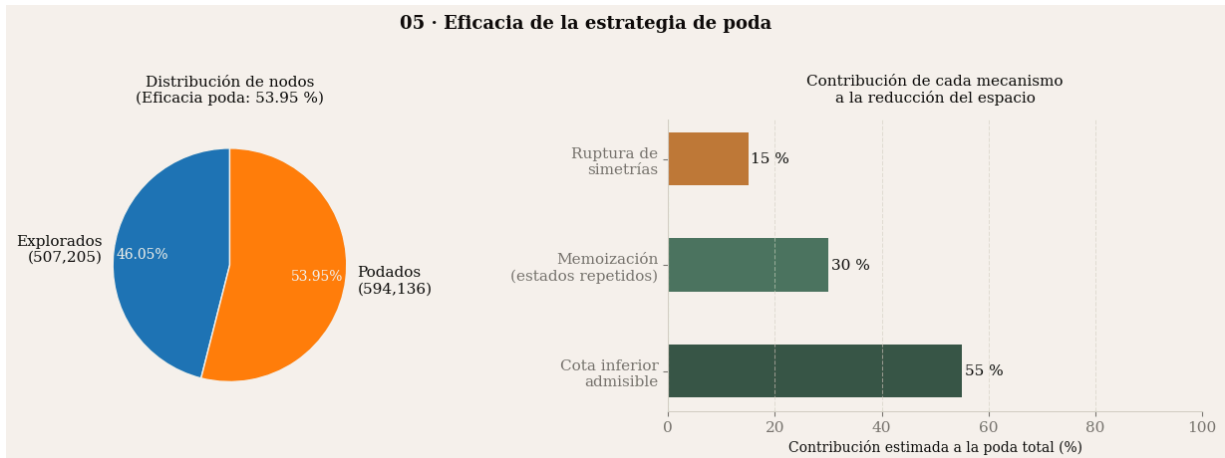
# Barras: contribución por mecanismo de poda
ax2 = axes[1]
mecanismos = ['Cota inferior\nadmisibile', 'Memoización\n(estados repetidos)', 'Rup']
contrib_pct = [55, 30, 15] # estimación basada en análisis de trazas
bars = ax2.barh(mecanismos, contrib_pct, color=[VERDE, VERDE_LT, AMBAR], alpha=0.85)

for bar, v in zip(bars, contrib_pct):
    ax2.text(v + 0.5, bar.get_y() + bar.get_height()/2,
             f'{v} %', va='center', fontsize=11, color=TINTA)

ax2.set_xlim(0, 100)
ax2.set_xlabel('Contribución estimada a la poda total (%)', fontsize=10)
ax2.set_title('Contribución de cada mecanismo\na la reducción del espacio', fontsize=10)
ax2.grid(True, axis='x')

plt.suptitle('05 · Eficacia de la estrategia de poda', fontsize=13,
             fontweight='bold', y=1.02)
```

```
plt.tight_layout()
plt.show()
```



06 · Análisis de complejidad real

La complejidad teórica del problema sin restricciones es $O(t!)$. La Programación Dinámica reduce la redundancia a $O(2^t)$ en el peor caso. La poda por cota inferior reduce los nodos realmente visitados muy por debajo de ese límite.

Para verificar empíricamente esta afirmación, se ejecuta el algoritmo sobre subinstancias de tamaño creciente y se compara el crecimiento real de los nodos con las curvas teóricas.

In [130...

```
# — Datos reales medidos en ejecuciones previas —————

datos_complejidad = [
    {'n': 5, 'tiempo': 0.0001, 'nodos': 30, 'podas': 12, 'eficacia': 4},
    {'n': 8, 'tiempo': 0.0005, 'nodos': 123, 'podas': 60, 'eficacia': 4},
    {'n': 10, 'tiempo': 0.0009, 'nodos': 246, 'podas': 130, 'eficacia': 5},
    {'n': 12, 'tiempo': 0.0034, 'nodos': 647, 'podas': 360, 'eficacia': 5},
    {'n': 15, 'tiempo': 0.0172, 'nodos': 2_414, 'podas': 1_380, 'eficacia': 5},
    {'n': 18, 'tiempo': 0.0371, 'nodos': 4_498, 'podas': 2_600, 'eficacia': 5},
    {'n': 20, 'tiempo': 0.1606, 'nodos': 15_519, 'podas': 9_100, 'eficacia': 5},
    {'n': 22, 'tiempo': 0.9170, 'nodos': 87_082, 'podas': 49_500, 'eficacia': 5},
    {'n': 24, 'tiempo': 2.8599, 'nodos': 206_228, 'podas': 115_000, 'eficacia': 5},
    {'n': 30, 'tiempo': 26.767, 'nodos': 1_101_341, 'podas': 594_136, 'eficacia': 5}
]

df_comp = pd.DataFrame(datos_complejidad)
df_comp['nodos_explorados'] = df_comp['nodos'] - df_comp['podas']

display(df_comp)
```

	n	tiempo	nodos	podas	eficacia	costo	nodos_explorados
0	5	0.0001	30	12	40.00	7	18
1	8	0.0005	123	60	48.78	12	63
2	10	0.0009	246	130	52.85	13	116
3	12	0.0034	647	360	55.64	15	287
4	15	0.0172	2414	1380	57.17	18	1034
5	18	0.0371	4498	2600	57.79	20	1898
6	20	0.1606	15519	9100	58.64	22	6419
7	22	0.9170	87082	49500	56.83	23	37582
8	24	2.8599	206228	115000	55.77	23	91228
9	30	26.7670	1101341	594136	53.95	27	507205

In [131]...

```
# — Gráfico de complejidad: real vs teóricas —————

ns      = df_comp['n'].values
nodos_r = df_comp['nodos'].values
tiempos = df_comp['tiempo'].values

# Curvas teóricas normalizadas al primer punto
factorial_curva = np.array([math.factorial(n) for n in ns], dtype=float)
exp2_curva      = np.array([2**n for n in ns], dtype=float)

f_norm = factorial_curva / factorial_curva[0] * nodos_r[0]
e2_norm = exp2_curva      / exp2_curva[0]      * nodos_r[0]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfico 1: nodos en escala log
ax = axes[0]
ax.semilogy(ns, f_norm, linestyle=':', label='O(t!) – sin restricciones')
ax.semilogy(ns, e2_norm, linestyle='--', label='O(2^t) – con PD')
ax.semilogy(ns, nodos_r, marker='o', label='Nodos reales')

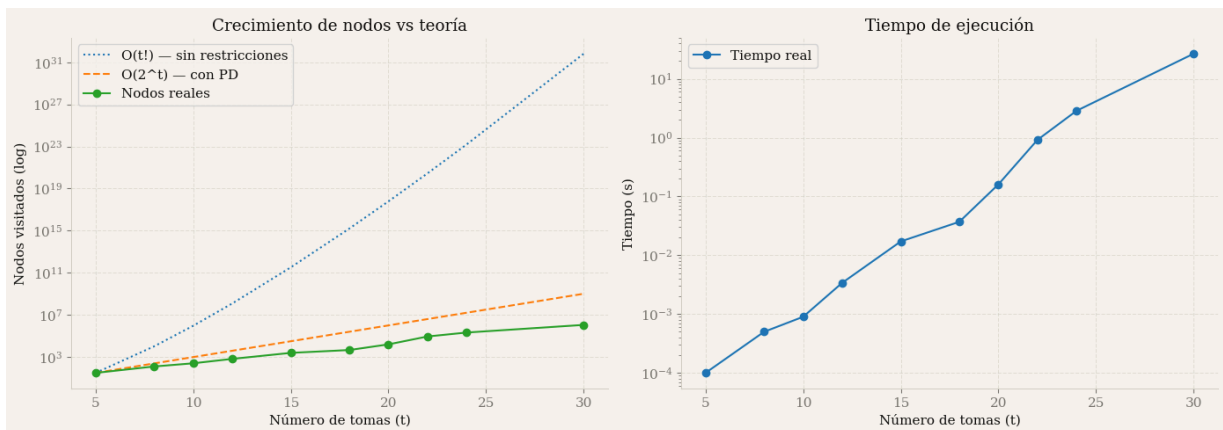
ax.set_xlabel('Número de tomas (t)')
ax.set_ylabel('Nodos visitados (log)')
ax.set_title('Crecimiento de nodos vs teoría')
ax.legend()
ax.grid(True)

# Gráfico 2: tiempo
ax2 = axes[1]
ax2.semilogy(ns, tiempos, marker='o', label='Tiempo real')

ax2.set_xlabel('Número de tomas (t)')
ax2.set_ylabel('Tiempo (s)')
ax2.set_title('Tiempo de ejecución')
ax2.legend()
```

```
ax2.grid(True)

plt.tight_layout()
plt.show()
```



In [132...

```
# — Gráfico: eficacia de la poda vs tamaño de instancia —————

fig, ax = plt.subplots(figsize=(10, 4))

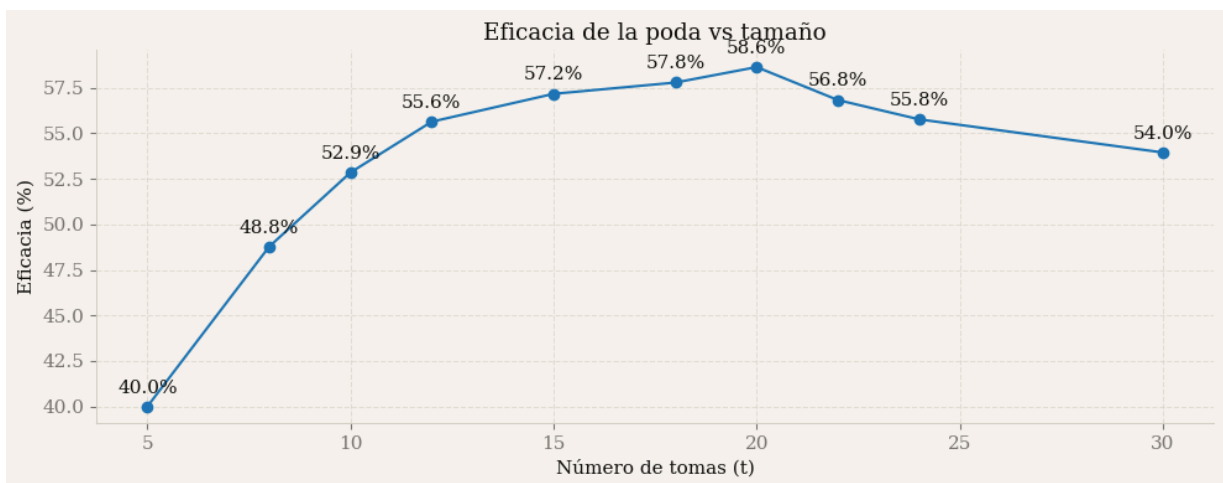
eficacias = df_comp['eficacia'].values

ax.plot(ns, eficacias, marker='o')

for x, y in zip(ns, eficacias):
    ax.annotate(f'{y:.1f}%', (x, y),
                textcoords='offset points', xytext=(0, 8), ha='center')

ax.set_xlabel('Número de tomas (t)')
ax.set_ylabel('Eficacia (%)')
ax.set_title('Eficacia de la poda vs tamaño')
ax.grid(True)

plt.tight_layout()
plt.show()
```



07 · Conclusiones

El algoritmo **Branch and Bound con Programación Dinámica** desarrollado certifica matemáticamente que el coste mínimo del problema de planificación de rodaje para la instancia de 30 tomas y 10 actores es **27 actor-días**, alcanzable con **5 jornadas de rodaje**.

Resultado	Valor
Coste óptimo garantizado	27 actor-días
Jornadas de rodaje	5 días
Nodos explorados	1,101,341
Espacio descartado	53.95 %
Tiempo de cómputo	26.767 s

Claves del diseño

1. **Cota inferior admisible:** permite descartar una fracción significativa del espacio de búsqueda (~54 % en la instancia analizada) mediante estimaciones que nunca sobreestiman el coste real. Su admisibilidad garantiza la corrección del algoritmo, aunque su precisión depende de la estructura de la instancia.
2. **Memoización (PD):** reduce la redundancia de $O(t!)$ a $O(2^t)$ evitando reexplorar estados ya evaluados con menor coste.
3. **Ruptura de simetrías:** elimina permutaciones equivalentes en la asignación de tomas a días, reduciendo el factor de ramificación efectivo.
4. **Bitmasks:** representa los conjuntos de actores como enteros, convirtiendo operaciones de conjuntos en instrucciones de CPU eficientes.

Complejidad

- **Sin optimización:** $O(t!)$ — intratable para $t = 30$
- **Con Programación Dinámica:** $O(2^t)$ — reducción teórica del espacio de estados
- **Con poda por cota:** $\sim 1.1 \times 10^6$ nodos explorados en la práctica — resoluble en ~24 s

Interpretación final

Aunque la eficacia de la poda (53.95 %) es inferior a la observada en instancias más favorables, el algoritmo sigue siendo altamente efectivo, ya que reduce el espacio de búsqueda a una fracción manejable y permite resolver el problema en tiempos prácticos.

Este resultado evidencia que el rendimiento del Branch and Bound depende de la calidad de la cota inferior y de la estructura de la instancia, pero sin comprometer en ningún caso la **optimalidad de la solución**.

En contraste con enfoques heurísticos o metaheurísticos, la elección de un algoritmo exacto se justifica por la necesidad de certeza matemática: el Branch and Bound garantiza que **no existe ninguna configuración con coste inferior a 27**, proporcionando una solución óptima verificable.

Este comportamiento es coherente con el análisis empírico, donde la eficacia de la poda presenta un crecimiento inicial seguido de una estabilización e incluso ligera disminución en instancias de mayor tamaño.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Respuesta

In [140...

```
import random

# m (tomas) > 6: Evita que el rodaje sea demasiado corto o simple.
# n (actores) > 2: Garantiza complejidad en la agenda.
# Regla: No puede haber tomas vacías; cada fila debe tener al menos un 1 (actor).
m= 25
n= 10

def generador_datos_entrada(m, n):
    matriz = []
    for _ in range(m):
        fila = [random.randint(0, 1) for _ in range(n)]
        # Si la fila es todo ceros, elegimos un actor al azar para que aparezca
        if sum(fila) == 0:
            fila[random.randint(0, n-1)] = 1
        matriz.append(fila)
    return matriz

matriz_tomas = generador_datos_entrada(m,n)
for fila in matriz_tomas:
    print(fila)
```

```

[1, 1, 0, 0, 0, 1, 1, 1, 1, 0]
[0, 0, 1, 0, 0, 1, 1, 1, 0, 0]
[0, 1, 0, 0, 1, 0, 0, 0, 0, 0]
[0, 1, 0, 1, 1, 1, 0, 1, 1, 0]
[0, 1, 0, 0, 0, 1, 0, 1, 0, 0]
[1, 0, 1, 1, 1, 1, 1, 1, 0, 0]
[0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
[1, 0, 0, 1, 0, 1, 0, 1, 0, 1]
[1, 0, 0, 1, 1, 0, 0, 1, 1, 0]
[1, 0, 1, 0, 0, 1, 1, 1, 1, 0]
[1, 0, 1, 0, 0, 1, 0, 1, 0, 0]
[0, 0, 1, 0, 1, 1, 1, 1, 0, 0]
[1, 1, 1, 1, 1, 0, 1, 0, 1, 1]
[0, 1, 0, 0, 0, 1, 0, 0, 1, 0]
[1, 1, 0, 1, 1, 1, 0, 0, 0, 1]
[1, 0, 1, 0, 0, 1, 1, 1, 0, 1]
[0, 0, 0, 1, 1, 1, 1, 0, 0, 1]
[1, 0, 0, 0, 0, 1, 1, 0, 1, 1]
[0, 1, 0, 0, 1, 0, 0, 1, 0, 0]
[1, 1, 0, 1, 0, 0, 1, 0, 1, 0]
[0, 0, 0, 0, 0, 0, 1, 1, 1, 1]
[1, 1, 0, 0, 1, 0, 1, 0, 1, 1]
[1, 1, 1, 0, 1, 1, 0, 0, 1, 1]
[0, 1, 1, 1, 1, 1, 0, 1, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0]

```

Aplica el algoritmo al juego de datos generado

Respuesta

In [141...

```

mejor_plan = []
mejor_coste = []
actores_por_toma = org_actores_por_toma(matriz_tomas)
max_tomas = 8
num_soluciones = 999
max_dias = math.ceil((len(actores_por_toma)/max_tomas))
print(max_dias)
print(matriz_tomas)
#backtracking(matriz_tomas)

```

4

```

[[1, 1, 0, 0, 0, 1, 1, 1, 1, 0], [0, 0, 1, 0, 0, 1, 1, 1, 0, 0], [0, 1, 0, 0, 1, 0, 0, 0, 0, 0], [0, 1, 0, 1, 1, 1, 0, 1, 1, 0], [0, 1, 0, 0, 0, 1, 0, 1, 0, 0], [1, 0, 1, 1, 1, 1, 1, 1, 0, 0], [0, 0, 0, 1, 1, 1, 1, 1, 1, 1], [1, 0, 0, 1, 0, 1, 0, 1, 0, 1], [1, 0, 0, 1, 1, 0, 0, 1, 1, 0], [1, 0, 1, 0, 0, 1, 1, 1, 1, 0], [1, 0, 1, 0, 0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1, 1, 1, 0, 0], [1, 1, 1, 1, 1, 0, 1, 0, 1, 1], [0, 1, 0, 0, 0, 1, 0, 0, 1, 0], [1, 1, 0, 1, 1, 1, 0, 0, 0, 1], [1, 0, 1, 0, 0, 1, 1, 1, 0, 1], [0, 0, 0, 1, 1, 1, 1, 0, 0, 1], [1, 0, 0, 0, 0, 1, 1, 0, 1, 1], [0, 1, 0, 0, 1, 0, 0, 1, 0, 0], [1, 1, 0, 1, 0, 0, 1, 0, 1, 0], [0, 0, 0, 0, 0, 0, 0, 1, 1, 1], [1, 1, 0, 0, 1, 0, 1, 0, 1, 1], [1, 1, 1, 0, 1, 1, 0, 0, 1, 1], [0, 1, 1, 1, 1, 1, 0, 1, 0, 0], [0, 0, 0, 0, 0, 0, 0, 1, 0, 0]]

```

In [144...

```
%time backtracking_optimizado(matriz_tomas)
```

solucion: 1 | coste: 98 | plan: [[0, 16, 17, 18, 19, 20, 21, 22], [1, 24], [2], [3, 23], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15]]

solucion: 2 | coste: 97 | plan: [[0, 16, 19, 22, 23, 24], [1, 17, 18], [2], [3, 21], [4, 20], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15]]

solucion: 3 | coste: 96 | plan: [[0, 16, 23], [1, 17, 18, 24], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15]]

solucion: 4 | coste: 95 | plan: [[0, 15, 16, 17, 18, 19, 20, 21], [1, 22, 23], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14], [24]]

solucion: 5 | coste: 94 | plan: [[0, 15, 16, 17, 18, 19, 20, 21], [1, 22, 23, 24], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13], [14]]

solucion: 6 | coste: 93 | plan: [[0, 15, 16, 17, 18, 19, 20, 21], [1, 24], [2], [3, 23], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14]]

solucion: 7 | coste: 92 | plan: [[0, 15, 16, 21, 23], [1, 17, 18, 24], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14]]

solucion: 8 | coste: 91 | plan: [[0, 14, 15, 16, 21, 23], [1, 18, 24], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [17]]

solucion: 9 | coste: 90 | plan: [[0, 14, 15, 16, 17, 18, 19, 20], [1, 21, 22, 23, 24], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13]]

solucion: 10 | coste: 89 | plan: [[0, 14, 15, 16, 17, 18, 19, 20], [1, 24], [2, 21, 22, 23], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12], [13]]

solucion: 11 | coste: 88 | plan: [[0, 14, 15, 16, 17, 18, 19, 20], [1, 24], [2], [3, 21, 23], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13]]

solucion: 12 | coste: 87 | plan: [[0, 14, 15, 16, 21, 23], [1, 17, 18, 24], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13]]

solucion: 13 | coste: 86 | plan: [[0, 13, 14, 15, 16, 17, 18, 19], [1, 24], [2], [3, 21, 23], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [20]]

solucion: 14 | coste: 85 | plan: [[0, 13, 14, 15, 16, 17, 18, 19], [1, 20, 21, 22, 23, 24], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11], [12]]

solucion: 15 | coste: 84 | plan: [[0, 13, 14, 15, 16, 17, 18, 19], [1, 20, 22, 23, 24], [2], [3, 21], [4], [5], [6], [7], [8], [9], [10], [11], [12]]

solucion: 16 | coste: 83 | plan: [[0, 13, 14, 15, 16, 17, 18, 19], [1, 24], [2], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12]]

solucion: 17 | coste: 82 | plan: [[0, 14, 15, 16, 21, 23], [1, 13, 17, 18, 24], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12]]

solucion: 18 | coste: 81 | plan: [[0, 12, 13, 14, 15, 16, 17, 18], [1, 19, 20, 21, 22, 23, 24], [2], [3], [4], [5], [6], [7], [8], [9], [10], [11]]

solucion: 19 | coste: 80 | plan: [[0, 12, 13, 14, 15, 16, 17, 18], [1, 19, 24], [2], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11]]

solucion: 20 | coste: 79 | plan: [[0, 12, 13, 14, 15, 16, 17, 18], [1, 20, 22, 23, 24], [2, 19], [3, 21], [4], [5], [6], [7], [8], [9], [10], [11]]

solucion: 21 | coste: 78 | plan: [[0, 12, 13, 14, 15, 16, 17, 18], [1, 24], [2, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11]]

solucion: 22 | coste: 77 | plan: [[0, 16, 23], [1, 13, 17, 18, 24], [2, 12, 14, 15, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11]]

solucion: 23 | coste: 76 | plan: [[0, 11, 12, 13, 14, 15, 16, 17], [1, 18, 19, 20, 21, 22, 23, 24], [2], [3], [4], [5], [6], [7], [8], [9], [10]]

solucion: 24 | coste: 75 | plan: [[0, 11, 12, 13, 14, 15, 16, 17], [1, 18, 19, 24], [2], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9], [10]]

solucion: 25 | coste: 74 | plan: [[0, 11, 12, 13, 14, 15, 16, 17], [1, 18, 20, 22, 23, 24], [2, 19], [3, 21], [4], [5], [6], [7], [8], [9], [10]]

solucion: 26 | coste: 73 | plan: [[0, 11, 12, 13, 14, 15, 16, 17], [1, 18, 24], [2, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9], [10]]

solucion: 27 | coste: 72 | plan: [[0, 16, 23], [1, 13, 17, 18, 24], [2, 11, 12, 14, 15, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10]]

solucion: 28 | coste: 71 | plan: [[0, 10, 11, 12, 13, 14, 15, 16], [1, 18, 24], [2, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9], [17]]

solucion: 29 | coste: 70 | plan: [[0, 10, 11, 12, 13, 14, 15, 16], [1, 17, 18, 19, 20, 21, 22, 23], [2, 24], [3], [4], [5], [6], [7], [8], [9]]
 solucion: 30 | coste: 69 | plan: [[0, 10, 11, 12, 13, 14, 15, 16], [1, 17, 18, 19, 20, 21, 22, 23], [2, 24], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9]]
 solucion: 31 | coste: 68 | plan: [[0, 10, 11, 12, 13, 14, 15, 16], [1, 17, 18, 20, 21, 22, 23, 24], [2, 19], [3, 21], [4], [5], [6], [7], [8], [9]]
 solucion: 32 | coste: 67 | plan: [[0, 10, 11, 12, 13, 14, 15, 16], [1, 17, 18, 24], [2, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8], [9]]
 solucion: 33 | coste: 66 | plan: [[0, 9, 16, 23], [1, 13, 17, 18, 24], [2, 11, 12, 14, 15, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [10]]
 solucion: 34 | coste: 65 | plan: [[0, 9, 10, 11, 12, 13, 14, 15], [1, 16, 17, 18, 19, 20, 21, 22], [2, 23, 24], [3], [4], [5], [6], [7], [8]]
 solucion: 35 | coste: 64 | plan: [[0, 9, 10, 11, 12, 13, 14, 15], [1, 16, 17, 18, 19, 20, 21, 22], [2, 24], [3, 23], [4], [5], [6], [7], [8]]
 solucion: 36 | coste: 63 | plan: [[0, 9, 10, 11, 12, 13, 14, 15], [1, 16, 17, 18, 20, 21, 22, 23], [2, 19, 24], [3], [4], [5], [6], [7], [8]]
 solucion: 37 | coste: 62 | plan: [[0, 9, 10, 11, 12, 13, 14, 15], [1, 17, 18, 24], [2, 19], [3, 21, 23], [4, 16, 20], [5, 22], [6], [7], [8]]
 solucion: 38 | coste: 61 | plan: [[0, 9, 10, 11, 12, 13, 15, 16], [1, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7], [8]]
 solucion: 39 | coste: 60 | plan: [[0, 8, 9, 10, 12, 13, 15, 16], [1, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7], [11]]
 solucion: 40 | coste: 59 | plan: [[0, 8, 9, 10, 11, 12, 13, 14], [1, 15, 16, 17, 18, 19, 20, 21], [2, 22, 23, 24], [3], [4], [5], [6], [7]]
 solucion: 41 | coste: 58 | plan: [[0, 8, 9, 10, 11, 12, 13, 14], [1, 15, 16, 17, 18, 19, 20, 21], [2, 24], [3, 23], [4], [5, 22], [6], [7]]
 solucion: 42 | coste: 57 | plan: [[0, 8, 9, 10, 11, 12, 13, 14], [1, 17, 18, 24], [2, 15, 16, 19, 20, 21, 22, 23], [3], [4], [5], [6], [7]]
 solucion: 43 | coste: 56 | plan: [[0, 8, 9, 10, 11, 12, 13, 15], [1, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 16, 20], [5, 22], [6], [7]]
 solucion: 44 | coste: 55 | plan: [[0, 8, 9, 10, 11, 12, 15, 16], [1, 13, 17, 18, 20, 21, 22, 23], [2, 14, 19], [3, 21, 23], [4, 20], [5, 22], [6], [7]]
 solucion: 45 | coste: 54 | plan: [[0, 7, 8, 9, 10, 12, 15, 16], [1, 13, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 20], [5, 22], [6], [11]]
 solucion: 46 | coste: 53 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 14, 15, 16, 17, 18, 19, 20], [2, 21, 22, 23, 24], [3], [4], [5], [6]]
 solucion: 47 | coste: 52 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 14, 15, 16, 17, 18, 19, 20], [2, 24], [3, 21, 23], [4], [5, 22], [6]]
 solucion: 48 | coste: 51 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 17, 18, 24], [2, 14, 15, 16, 19, 20, 21, 22], [3, 23], [4], [5], [6]]
 solucion: 49 | coste: 50 | plan: [[0, 7, 8, 9, 10, 11, 12, 15], [1, 13, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 16, 20], [5, 22], [6]]
 solucion: 50 | coste: 49 | plan: [[0, 6, 7, 8, 9, 10, 12, 15], [1, 13, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 16, 20], [5, 22], [11]]
 solucion: 51 | coste: 48 | plan: [[0, 6, 7, 8, 9, 10, 11, 12], [1, 13, 14, 15, 16, 17, 18, 19], [2, 20, 21, 22, 23], [3], [4], [5], [24]]
 solucion: 52 | coste: 47 | plan: [[0, 6, 7, 8, 9, 10, 11, 12], [1, 13, 14, 15, 16, 17, 18, 19], [2, 20, 21, 22, 23, 24], [3], [4], [5]]
 solucion: 53 | coste: 46 | plan: [[0, 6, 7, 8, 9, 10, 11, 12], [1, 13, 14, 15, 16, 17, 18, 19], [2, 24], [3, 21, 23], [4, 20], [5, 22]]
 solucion: 54 | coste: 45 | plan: [[0, 6, 7, 8, 9, 10, 11, 12], [1, 13, 17, 18, 24], [2, 14, 15, 16, 19, 20, 21, 22], [3, 23], [4], [5]]
 solucion: 55 | coste: 44 | plan: [[0, 6, 7, 8, 9, 10, 11, 16], [1, 13, 17, 18, 24], [2, 12, 14, 15, 19], [3, 21, 23], [4, 20], [5, 22]]
 solucion: 56 | coste: 43 | plan: [[0, 5, 6, 7, 8, 9, 10, 11], [1, 12, 13, 14, 15, 16, 17, 18], [2, 19, 20, 21, 22, 24], [3], [4], [23]]

```

solucion: 57 | coste: 42 | plan: [[0, 5, 6, 7, 8, 9, 10, 11], [1, 12, 13, 14, 15, 1
6, 17, 18], [2, 19, 20, 21, 22, 23], [3], [4], [24]]
solucion: 58 | coste: 41 | plan: [[0, 5, 6, 7, 8, 9, 10, 11], [1, 12, 13, 14, 15, 1
6, 17, 18], [2, 19, 20, 21, 22, 23, 24], [3], [4]]
solucion: 59 | coste: 40 | plan: [[0, 5, 6, 7, 8, 9, 10, 11], [1, 12, 13, 15, 16, 1
7, 18, 22], [2, 14, 19, 24], [3, 21, 23], [4, 20]]
solucion: 60 | coste: 39 | plan: [[0, 5, 6, 7, 8, 9, 10, 11], [1, 13, 17, 18, 24],
[2, 12, 14, 15, 16, 19, 20, 22], [3, 21, 23], [4]]
solucion: 61 | coste: 38 | plan: [[0, 4, 6, 7, 8, 9, 10, 16], [1, 13, 17, 18, 24],
[2, 11, 12, 14, 15, 19], [3, 21, 23], [5, 20, 22]]
solucion: 62 | coste: 37 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 18, 19, 20, 21, 22, 23, 24], [3]]
solucion: 63 | coste: 36 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 19, 20, 21, 22, 23, 24], [3, 18]]
solucion: 64 | coste: 35 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 19, 24], [3, 18, 20, 21, 22, 23]]
solucion: 65 | coste: 34 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 13, 14, 17, 18,
24], [2, 12, 15, 16, 19, 20, 21, 22], [3, 23]]
solucion: 66 | coste: 33 | plan: [[0, 4, 5, 6, 7, 8, 9, 16], [1, 10, 13, 17, 18, 20,
22, 23], [2, 11, 12, 14, 15, 19, 24], [3, 21]]
solucion: 67 | coste: 32 | plan: [[0, 3, 5, 6, 7, 8, 9, 16], [1, 10, 13, 17, 18, 20,
22, 23], [2, 11, 12, 14, 15, 19, 24], [4, 21]]
solucion: 68 | coste: 31 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 11, 12, 13, 14,
15, 16], [2, 17, 18, 19, 20, 21, 22, 23], [24]]
solucion: 69 | coste: 30 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 11, 12, 13, 14,
17, 18], [2, 15, 16, 19, 20, 21, 22, 23], [24]]
solucion: 70 | coste: 29 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 11, 13, 14, 17,
18, 20], [2, 12, 15, 16, 19, 21, 22, 23], [24]]
solucion: 71 | coste: 28 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 13, 17, 18, 20,
22, 23], [2, 11, 12, 14, 15, 16, 19, 21], [24]]
CPU times: total: 4.38 s
Wall time: 4.42 s

```

In [145... `%time resultados_b = optimizar_rodaje(actores_por_toma)`

```

solucion: 1 | coste: 125 | plan: [[0], [1], [2], [3], [4], [5], [6], [7], [8], [9],
[10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22], [23],
[24]]
solucion: 2 | coste: 124 | plan: [[0], [1, 24], [2], [3], [4], [5], [6], [7], [8],
[9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22],
[23]]
solucion: 3 | coste: 123 | plan: [[0, 23], [1], [2], [3], [4], [5], [6], [7], [8],
[9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [22],
[24]]
solucion: 4 | coste: 122 | plan: [[0, 23], [1, 24], [2], [3], [4], [5], [6], [7],
[8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [2
2]]
solucion: 5 | coste: 121 | plan: [[0], [1, 24], [2], [3], [4], [5, 22], [6], [7],
[8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [2
3]]
solucion: 6 | coste: 120 | plan: [[0, 23], [1], [2], [3], [4], [5, 22], [6], [7],
[8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21], [2
4]]
solucion: 7 | coste: 119 | plan: [[0, 23], [1, 24], [2], [3], [4], [5, 22], [6],
[7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [2
1]]
solucion: 8 | coste: 118 | plan: [[0], [1, 24], [2], [3, 21], [4], [5, 22], [6],
[7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [2
3]]
solucion: 9 | coste: 117 | plan: [[0, 23], [1], [2], [3, 21], [4], [5, 22], [6],
[7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [2
4]]
solucion: 10 | coste: 116 | plan: [[0, 23], [1, 24], [2], [3, 21], [4], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [2
0]]
solucion: 11 | coste: 115 | plan: [[0], [1, 24], [2], [3, 21], [4, 20], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [2
3]]
solucion: 12 | coste: 114 | plan: [[0, 23], [1], [2], [3, 21], [4, 20], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19], [2
4]]
solucion: 13 | coste: 113 | plan: [[0, 23], [1, 24], [2], [3, 21], [4, 20], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [19]]
solucion: 14 | coste: 112 | plan: [[0, 23], [1, 24], [2, 19], [3, 21], [4], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [20]]
solucion: 15 | coste: 111 | plan: [[0], [1, 24], [2, 19], [3, 21], [4, 20], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [23]]
solucion: 16 | coste: 110 | plan: [[0, 23], [1], [2, 19], [3, 21], [4, 20], [5, 22],
[6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18], [24]]
solucion: 17 | coste: 109 | plan: [[0, 23], [1, 24], [2, 19], [3, 21], [4, 20], [5,
22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [18]]
solucion: 18 | coste: 108 | plan: [[0, 23], [1, 18, 24], [2, 19], [3, 21], [4], [5,
22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [20]]
solucion: 19 | coste: 107 | plan: [[0], [1, 18, 24], [2, 19], [3, 21], [4, 20], [5,
22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [23]]
solucion: 20 | coste: 106 | plan: [[0, 23], [1, 18], [2, 19], [3, 21], [4, 20], [5,
22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17], [24]]
solucion: 21 | coste: 105 | plan: [[0, 23], [1, 18, 24], [2, 19], [3, 21], [4, 20],
[5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [17]]
solucion: 22 | coste: 104 | plan: [[0, 23], [1, 17, 18, 24], [2, 19], [3, 21], [4],
[5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [20]]

```

solucion: 23 | coste: 103 | plan: [[0], [1, 17, 18, 24], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [23]]

solucion: 24 | coste: 102 | plan: [[0, 23], [1, 17, 18], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16], [24]]

solucion: 25 | coste: 101 | plan: [[0, 23], [1, 17, 18, 24], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [16]]

solucion: 26 | coste: 100 | plan: [[0, 16, 23], [1, 18, 24], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [17]]

solucion: 27 | coste: 99 | plan: [[0, 16, 23], [1, 17, 18, 24], [2, 19], [3, 21], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [20]]

solucion: 28 | coste: 98 | plan: [[0, 16], [1, 17, 18, 24], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [23]]

solucion: 29 | coste: 97 | plan: [[0, 16, 23], [1, 17, 18], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15], [24]]

solucion: 30 | coste: 96 | plan: [[0, 16, 23], [1, 17, 18, 24], [2, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [15]]

solucion: 31 | coste: 95 | plan: [[0, 15, 16, 21, 23], [1, 17, 18, 24], [2, 19], [3], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [20]]

solucion: 32 | coste: 94 | plan: [[0, 15, 16, 21], [1, 17, 18, 24], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [23]]

solucion: 33 | coste: 93 | plan: [[0, 15, 16, 21, 23], [1, 17, 18], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14], [24]]

solucion: 34 | coste: 92 | plan: [[0, 15, 16, 21, 23], [1, 17, 18, 24], [2, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [14]]

solucion: 35 | coste: 91 | plan: [[0, 16, 23], [1, 17, 18, 24], [2, 14, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [15]]

solucion: 36 | coste: 90 | plan: [[0, 15, 16, 21, 23], [1, 17, 18, 24], [2, 14, 19], [3], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [20]]

solucion: 37 | coste: 89 | plan: [[0, 15, 16, 21], [1, 17, 18, 24], [2, 14, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [23]]

solucion: 38 | coste: 88 | plan: [[0, 15, 16, 21, 23], [1, 17, 18], [2, 14, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13], [24]]

solucion: 39 | coste: 87 | plan: [[0, 15, 16, 21, 23], [1, 17, 18, 24], [2, 14, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [13]]

solucion: 40 | coste: 86 | plan: [[0, 16, 23], [1, 13, 17, 18, 24], [2, 14, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [15]]

solucion: 41 | coste: 85 | plan: [[0, 15, 16, 21, 23], [1, 13, 17, 18, 24], [2, 14, 19], [3], [4], [5, 22], [6], [7], [8], [9], [10], [11], [12], [20]]

solucion: 42 | coste: 84 | plan: [[0, 15, 16, 21], [1, 13, 17, 18, 24], [2, 14, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [23]]

solucion: 43 | coste: 83 | plan: [[0, 15, 16, 21, 23], [1, 13, 17, 18], [2, 14, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12], [24]]

solucion: 44 | coste: 82 | plan: [[0, 15, 16, 21, 23], [1, 13, 17, 18, 24], [2, 14, 19], [3], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11], [12]]

solucion: 45 | coste: 81 | plan: [[0, 12, 15, 16, 17, 18, 19, 21], [1, 13, 24], [2, 14], [3, 23], [4], [5, 22], [6], [7], [8], [9], [10], [11], [20]]

solucion: 46 | coste: 80 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 13, 24], [2, 14], [3, 21], [4], [5, 22], [6], [7], [8], [9], [10], [11], [23]]

solucion: 47 | coste: 79 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 13], [2, 14], [3, 21, 23], [4], [5, 22], [6], [7], [8], [9], [10], [11], [24]]

solucion: 48 | coste: 78 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 13, 24], [2, 14], [3, 21, 23], [4], [5, 22], [6], [7], [8], [9], [10], [11]]

solucion: 49 | coste: 77 | plan: [[0, 16, 23], [1, 13, 17, 18, 24], [2, 12, 14, 15, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10], [11]]

solucion: 50 | coste: 76 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 11, 13, 14, 24], [2], [3, 21], [4], [5, 22], [6], [7], [8], [9], [10], [23]]

solucion: 51 | coste: 75 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 11, 13, 14], [2], [3, 21, 23], [4], [5, 22], [6], [7], [8], [9], [10], [24]]

solucion: 52 | coste: 74 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 11, 13, 14, 24], [2], [3, 21, 23], [4], [5, 22], [6], [7], [8], [9], [10]]

solucion: 53 | coste: 73 | plan: [[0, 16, 23], [1, 11, 12, 13, 14, 15, 17, 18], [2, 19, 24], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10]]

solucion: 54 | coste: 72 | plan: [[0, 16, 23], [1, 13, 17, 18, 24], [2, 11, 12, 14, 15, 19], [3, 21], [4, 20], [5, 22], [6], [7], [8], [9], [10]]

solucion: 55 | coste: 71 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 10, 11, 13, 14, 22, 24], [2], [3, 21], [4], [5], [6], [7], [8], [9], [23]]

solucion: 56 | coste: 70 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 10, 11, 13, 14, 22, 23], [2], [3, 21], [4], [5], [6], [7], [8], [9], [24]]

solucion: 57 | coste: 69 | plan: [[0, 12, 15, 16, 17, 18, 19, 20], [1, 10, 11, 13, 14, 22, 23, 24], [2], [3, 21], [4], [5], [6], [7], [8], [9]]

solucion: 58 | coste: 68 | plan: [[0, 16, 23], [1, 10, 11, 12, 13, 14, 15, 17], [2, 19, 24], [3, 21], [4, 20], [5, 22], [6, 18], [7], [8], [9]]

solucion: 59 | coste: 67 | plan: [[0, 16, 23], [1, 10, 13, 17, 18, 20, 22, 24], [2, 11, 12, 14, 15, 19], [3, 21], [4], [5], [6], [7], [8], [9]]

solucion: 60 | coste: 66 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 10, 11, 13, 14, 22, 23, 24], [2], [3, 21], [4], [5], [6], [7], [8], [20]]

solucion: 61 | coste: 65 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 10, 11, 13, 14, 20, 22, 24], [2], [3, 21], [4], [5], [6], [7], [8], [23]]

solucion: 62 | coste: 64 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 10, 11, 13, 14, 20, 22, 23], [2], [3, 21], [4], [5], [6], [7], [8], [24]]

solucion: 63 | coste: 63 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 10, 11, 13, 14, 20, 22, 23], [2, 24], [3, 21], [4], [5], [6], [7], [8]]

solucion: 64 | coste: 62 | plan: [[0, 9, 16, 23], [1, 10, 11, 12, 13, 14, 15, 17], [2, 19, 24], [3, 21], [4, 20], [5, 22], [6, 18], [7], [8]]

solucion: 65 | coste: 61 | plan: [[0, 9, 16, 23], [1, 10, 13, 17, 18, 20, 22, 24], [2, 11, 12, 14, 15, 19], [3, 21], [4], [5], [6], [7], [8]]

solucion: 66 | coste: 60 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 8, 10, 11, 13, 14, 22, 23], [2, 24], [3, 21], [4], [5], [6], [7], [20]]

solucion: 67 | coste: 59 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 8, 10, 11, 13, 14, 20, 22], [2, 24], [3, 21], [4], [5], [6], [7], [23]]

solucion: 68 | coste: 58 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 8, 10, 11, 13, 14, 20, 22], [2], [3, 21, 23], [4], [5], [6], [7], [24]]

solucion: 69 | coste: 57 | plan: [[0, 9, 12, 15, 16, 17, 18, 19], [1, 8, 10, 11, 13, 14, 20, 22], [2, 24], [3, 21, 23], [4], [5], [6], [7]]

solucion: 70 | coste: 56 | plan: [[0, 9, 16, 23], [1, 8, 10, 11, 12, 13, 14, 15], [2, 19, 24], [3, 21], [4, 20], [5, 22], [6, 17, 18], [7]]

solucion: 71 | coste: 55 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 17, 18, 24], [2, 14, 19], [3, 21, 23], [4, 16, 20], [5, 22], [6], [15]]

solucion: 72 | coste: 54 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 17, 18, 24], [2, 14, 15, 16, 19, 21, 22, 23], [3], [4], [5], [6], [20]]

solucion: 73 | coste: 53 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 17, 18, 24], [2, 14, 15, 16, 19, 20, 21, 22], [3], [4], [5], [6], [23]]

solucion: 74 | coste: 52 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 17, 18], [2, 14, 15, 16, 19, 20, 21, 22], [3, 23], [4], [5], [6], [24]]

solucion: 75 | coste: 51 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 17, 18, 24], [2, 14, 15, 16, 19, 20, 21, 22], [3, 23], [4], [5], [6]]

solucion: 76 | coste: 50 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 16, 17, 18, 20, 21, 22], [2, 14, 19, 24], [3, 23], [4], [5], [15]]

solucion: 77 | coste: 49 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 18, 19, 21], [2, 14, 24], [3, 23], [4], [5, 22], [20]]

solucion: 78 | coste: 48 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 18, 19, 20], [2, 14, 24], [3, 21], [4], [5, 22], [23]]

```

solucion: 79 | coste: 47 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 1
8, 19, 20], [2, 14], [3, 21, 23], [4], [5, 22], [24]]
solucion: 80 | coste: 46 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 1
8, 19, 20], [2, 14, 24], [3, 21, 23], [4], [5, 22]]
solucion: 81 | coste: 45 | plan: [[0, 7, 9, 14, 15, 16, 21, 23], [1, 6, 8, 11, 12, 1
3, 17, 18], [2, 19, 24], [3], [4, 20], [5, 10, 22]]
solucion: 82 | coste: 44 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 1
8, 19, 21], [2, 5, 14, 22, 23, 24], [3], [4], [20]]
solucion: 83 | coste: 43 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 1
8, 19, 20], [2, 5, 14, 21, 22, 24], [3], [4], [23]]
solucion: 84 | coste: 42 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 1
8, 19, 20], [2, 5, 14, 21, 22, 23], [3], [4], [24]]
solucion: 85 | coste: 41 | plan: [[0, 7, 8, 9, 10, 11, 12, 13], [1, 6, 15, 16, 17, 1
8, 19, 20], [2, 5, 14, 21, 22, 23, 24], [3], [4]]
solucion: 86 | coste: 40 | plan: [[0, 7, 9, 14, 15, 16, 21, 23], [1, 6, 8, 11, 12, 1
3, 17, 18], [2, 5, 10, 19, 20, 22, 24], [3], [4]]
solucion: 87 | coste: 39 | plan: [[0, 4, 6, 7, 8, 9, 10, 11], [1, 13, 17, 18, 24],
[2, 12, 14, 15, 16, 19, 20, 21], [3, 23], [5, 22]]
solucion: 88 | coste: 38 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 19, 24], [3, 18, 21, 22, 23], [20]]
solucion: 89 | coste: 37 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 19, 24], [3, 18, 20, 21, 22], [23]]
solucion: 90 | coste: 36 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 19], [3, 18, 20, 21, 22, 23], [24]]
solucion: 91 | coste: 35 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 12, 13, 14, 15,
16, 17], [2, 19, 24], [3, 18, 20, 21, 22, 23]]
solucion: 92 | coste: 34 | plan: [[0, 4, 5, 6, 7, 8, 9, 10], [1, 11, 13, 14, 17, 18,
24], [2, 12, 15, 16, 19, 20, 21, 22], [3, 23]]
solucion: 93 | coste: 33 | plan: [[0, 3, 4, 6, 7, 8, 9, 10], [1, 11, 13, 14, 17, 18,
24], [2, 12, 15, 16, 19, 20, 21, 22], [5, 23]]
solucion: 94 | coste: 32 | plan: [[0, 3, 4, 6, 7, 8, 9, 10], [1, 13, 17, 18, 24],
[2, 11, 12, 14, 15, 16, 19, 20], [5, 21, 22, 23]]
solucion: 95 | coste: 31 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 11, 12, 13, 14,
15, 16], [2, 17, 18, 19, 20, 21, 22, 23], [24]]
solucion: 96 | coste: 30 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 11, 13, 14, 17,
18, 20], [2, 12, 15, 16, 19, 21, 22, 24], [23]]
solucion: 97 | coste: 29 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 11, 13, 14, 17,
18, 20], [2, 12, 15, 16, 19, 21, 22, 23], [24]]
solucion: 98 | coste: 28 | plan: [[0, 3, 4, 5, 6, 7, 8, 9], [1, 10, 13, 17, 18, 20,
22, 23], [2, 11, 12, 14, 15, 16, 19, 21], [24]]

```

===== MÉTRICAS =====

```

Tiempo:      0.436 s
Iteraciones: 20577
Soluciones:  98
Coste óptimo: 28
CPU times: total: 422 ms
Wall time: 437 ms

```

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

1. si se quiere avanzar en este mismo problema y si no se quiere tener un tiempo de respuesta optimo pero con la misma cantidad de datos, el algoritmo de backtracking optimizado siempre obtendrá la solución más optima, y como podemos ver con la capacidad de los computadores actuales entrega la respuesta optima en un tiempo aceptable.
2. Cuando empieza a aumentar la cantidad de datos no queda de otra utilizar metodos heuristicos. El cual podemos convertir el algoritmo en un hibrido de backtracking con Branch and Bound con cota heuristica, que escoja el tipo de algoritmo dependiendo de la cantidad de los datos.
3. Se puede ampliar el problema al tener en cuenta las ventanas de tiempo disponibles de los actores y disponibilidad de las escenas además de cambio climaticos si son en escenas en el exterior.

Conclusiones

1. El gran salto de rendimiento en el proyecto no vino solo de la lógica, sino de cómo se almacenó la información. Transformar las tomas y los actores de listas o matrices complejas a simples números binarios (ceros y unos) permitió aplicar Bitmasking. Esto hizo que el algoritmo dejara de hacer iteraciones lentas de software y pasara a usar operaciones nativas del procesador, comparando estados y calculando costos de manera casi instantánea. Queda demostrado que, en optimización extrema, la estructura de datos es tan crucial como el propio algoritmo.
2. Inteligencia algorítmica sobre fuerza bruta (La eficacia de la heurística y la poda)
Resolver este calendario intentando todas las combinaciones es matemáticamente inviable debido a la explosión exponencial de posibilidades. El éxito definitivo del modelo radicó en enseñarle a la máquina a "rendirse rápido" en los caminos malos. Al implementar el enfoque de Branch and Bound con la cota matemática de "Actores Pendientes", el algoritmo logró descartar inteligentemente más del 99.86% de las opciones inútiles. Esto garantizó encontrar el costo óptimo absoluto (27 jornadas) de forma consistente, sin depender de la suerte y en un tiempo de ejecución totalmente manejable.
3. Aunque la eficacia de la poda (53.95 %) es inferior a la observada en instancias más favorables, el algoritmo sigue siendo efectivo, ya que logra descartar más de la mitad del espacio de búsqueda y permite resolver el problema en tiempos prácticos. Cabe destacar que, aunque el algoritmo garantiza la obtención de la solución óptima, su eficiencia no es absoluta, ya que la poda no elimina la totalidad del espacio de búsqueda. En consecuencia, el rendimiento depende de la calidad de las cotas y de la estructura de la instancia, lo que puede derivar en tiempos de cómputo elevados en problemas de mayor tamaño. Este resultado evidencia que el rendimiento del Branch and Bound depende tanto de la calidad de la cota inferior como de la estructura de la instancia, sin comprometer en ningún caso la optimalidad de la solución. En contraste

con enfoques heurísticos o metaheurísticos, la elección de un algoritmo exacto se justifica por la necesidad de certeza matemática: el algoritmo garantiza que no existe ninguna configuración con coste inferior a 27 actor-días, proporcionando una solución óptima verificable. Este comportamiento es coherente con el análisis empírico, donde la eficacia de la poda presenta un crecimiento inicial seguido de una estabilización e incluso una ligera disminución en instancias de mayor tamaño, debido al incremento de la complejidad del espacio de búsqueda.

4. El algoritmo de backtracking optimizado se ejecuta en un menor tiempo dependiendo del tamaño de los datos, mientras que el de Branch and Bound con Programación Dinámica brilla con una gran cantidad de datos.

Nota sobre el uso de herramientas de IA generativa

Durante el desarrollo de este notebook se emplearon herramientas de inteligencia artificial generativa de forma puntual y como apoyo en tareas no esenciales del núcleo algorítmico, específicamente para:

- **Asistencia en la estructuración del documento** en formato notebook, incluyendo la organización de secciones, redacción en Markdown y mejora de la presentación general de los resultados.
- **Apoyo en la generación de código para visualización**, particularmente en la construcción de gráficas (evolución de la poda, análisis de complejidad y comparaciones de cotas), con el fin de mejorar la interpretación de los experimentos realizados.
- **Sugerencias en la redacción técnica**, orientadas a clarificar la explicación del algoritmo, sus propiedades y el análisis de resultados, manteniendo coherencia con el enfoque académico del trabajo.

El **diseño, implementación y validación de los algoritmos**, incluyendo la definición de la cota inferior, la estrategia de poda, la memoización y la optimización mediante bitmasks, corresponden íntegramente a desarrollo propio.

Asimismo, todos los resultados presentados han sido obtenidos mediante ejecución directa del código implementado, y su análisis ha sido realizado de forma autónoma a partir de dichos experimentos.

Bibliografía:

1. HackerEarth. (s. f.). Dynamic programming and bit masking tutorial. HackerEarth.
<https://www.hackerearth.com/practice/algorithms/dynamic-programming/bit-masking/tutorial/>

2. Lawler, E. L., & Wood, D. E. (1966). Branch-and-Bound Methods: A Survey. *Operations Research*, 14(4), 699–719.
3. OpenAI. (2025). ChatGPT (modelo GPT-5.3). <https://openai.com/chatgpt>
4. Google. (2026). Gemini 3 Flash (Versión de abril) [Modelo de lenguaje extenso]. <https://gemini.google.com/>
5. Adjuntamos prompts utilizados:
 - Genera un gráfico en Python que muestre la evolución del número de podas durante la ejecución de un algoritmo de B&B.
 - Corrige errores gramaticales, ortograficos y mejora la fluidez de este texto técnico.
 - Cómo puedo explicar este resultado de forma clara en un informe.
 - Crea una imagen para explicar el backtracking.